

# **Ray Tracing II:**

## **Physically based rendering with path tracing**

# Logistics

---

- Assignment 1 grading finished
  - For the future, gcc
- Remote lab sessions
  - B09: Tuesday 3:30-4:20 pm
  - B11: Wednesday 10:30-11:20 am

# Where we left off

---

$$L = k_a I_a + k_d I \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s I \max(0, \mathbf{n} \cdot \mathbf{h})^p$$



Ambient



Diffuse

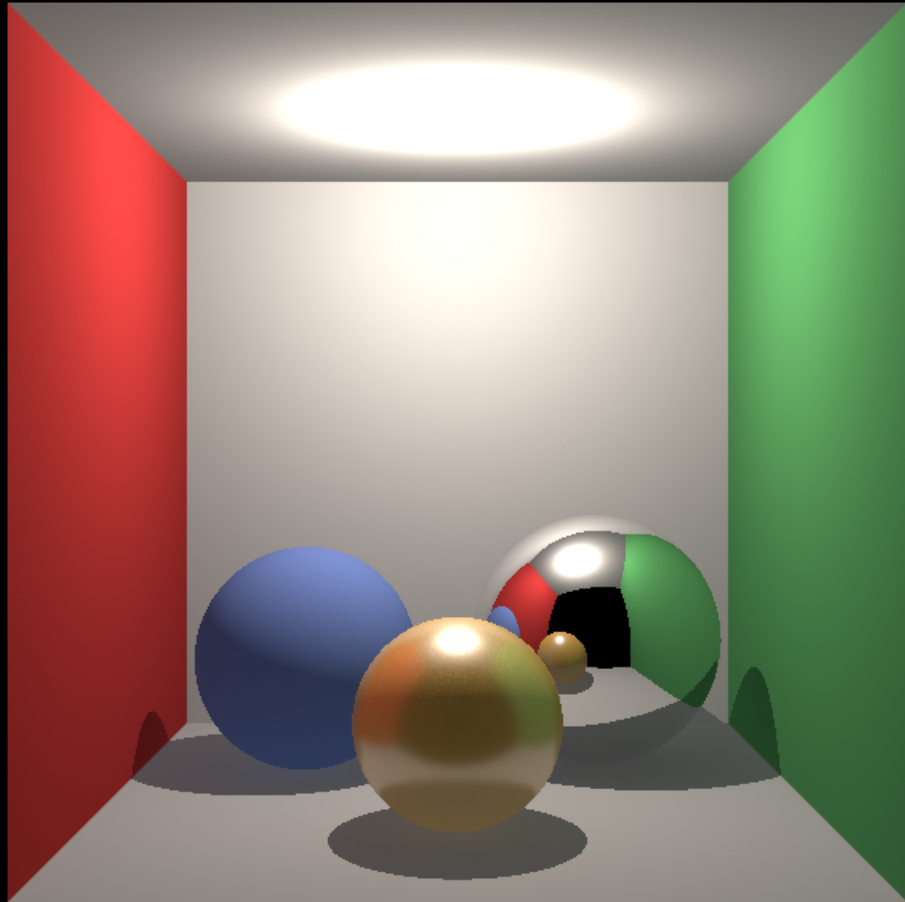


Specular

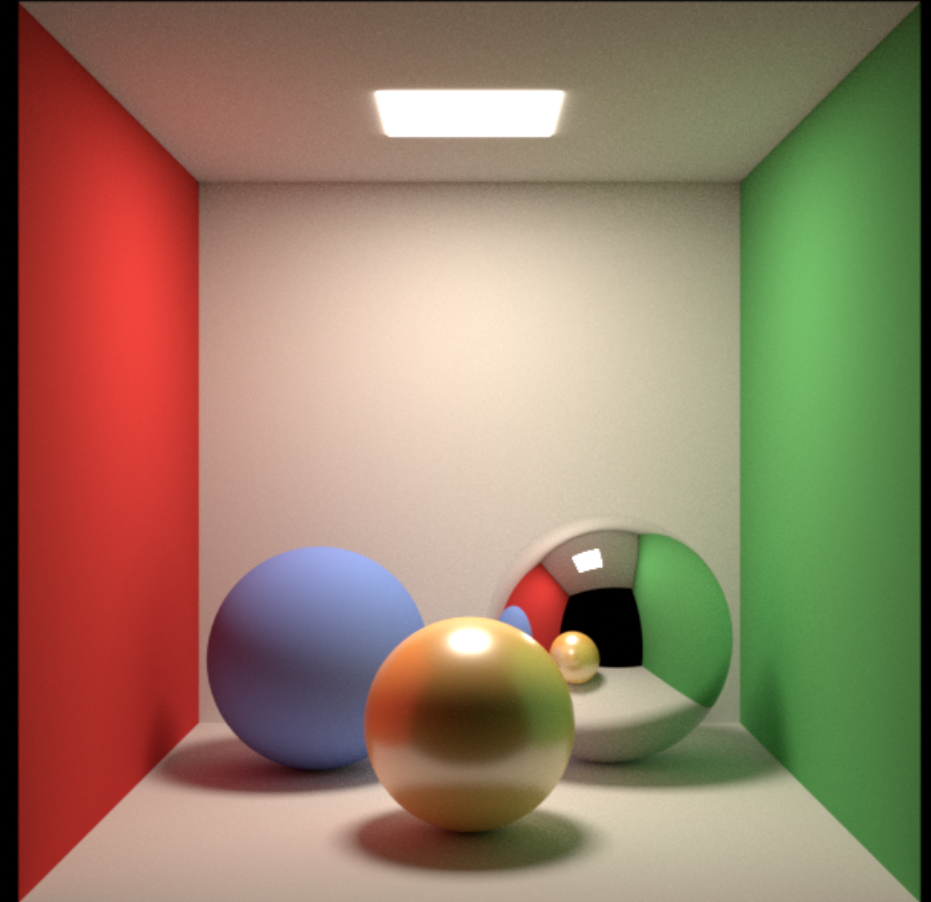
- + shadow rays, + mirror recursion
- Non-physically based

# What is missing?

---



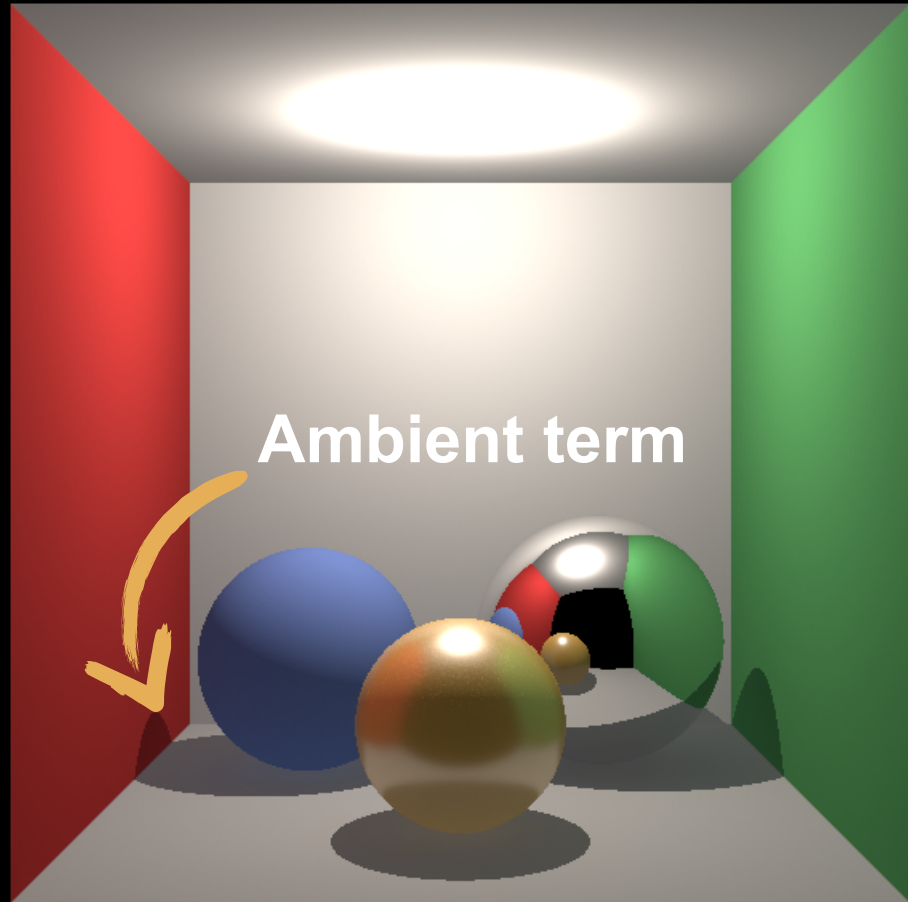
**Last lecture: Non-physically based**



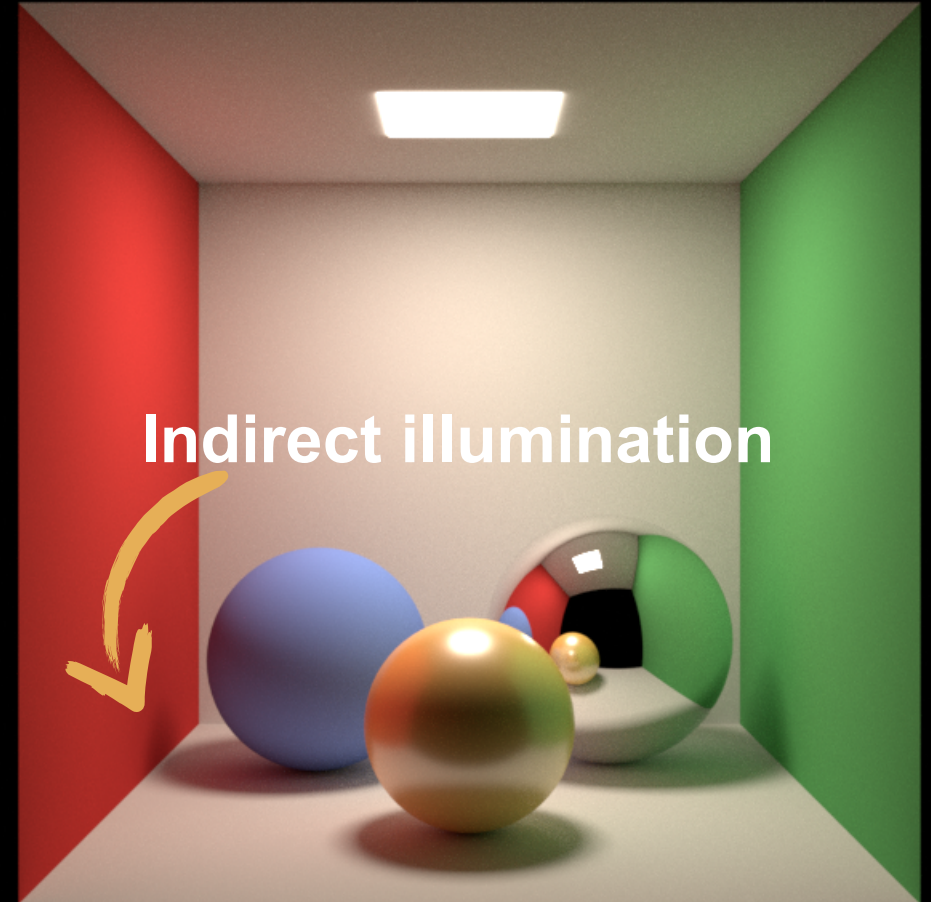
**This lecture: Physically based**

# What is missing?

---



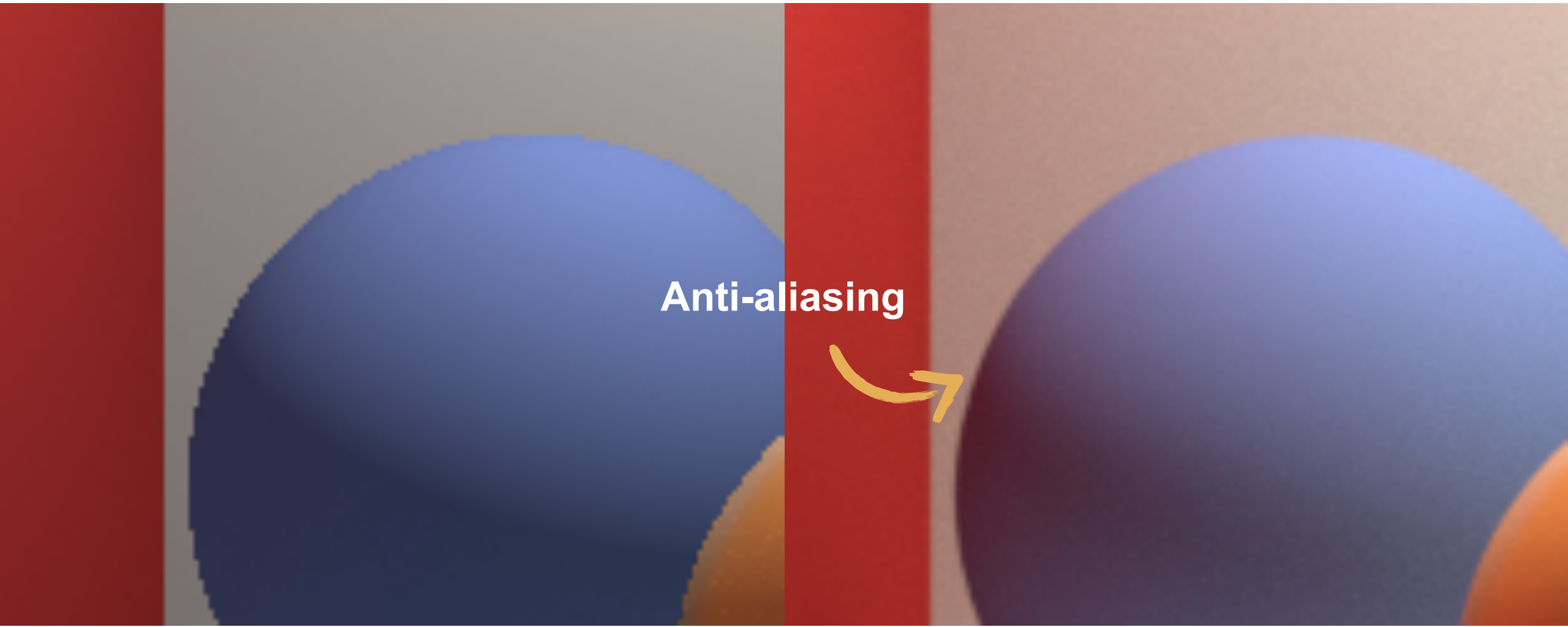
**Last lecture: Non-physically based**



**This lecture: Physically based**

# What is missing?

---



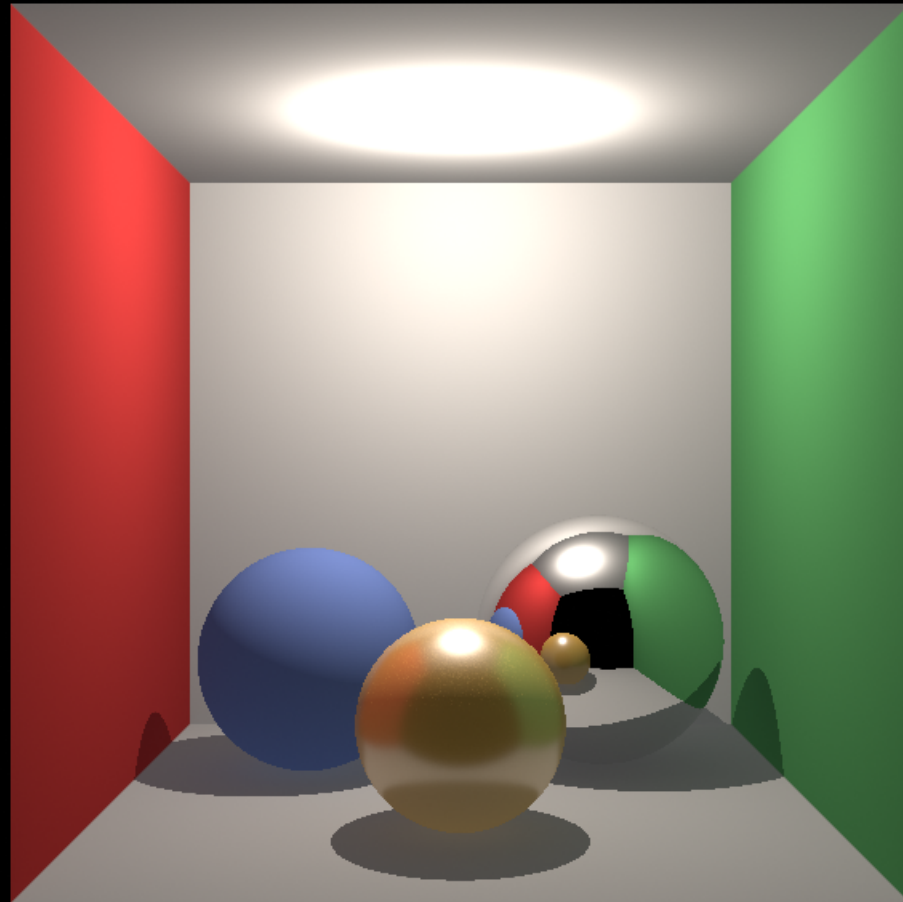
Anti-aliasing

**Last lecture: Non-physically based**

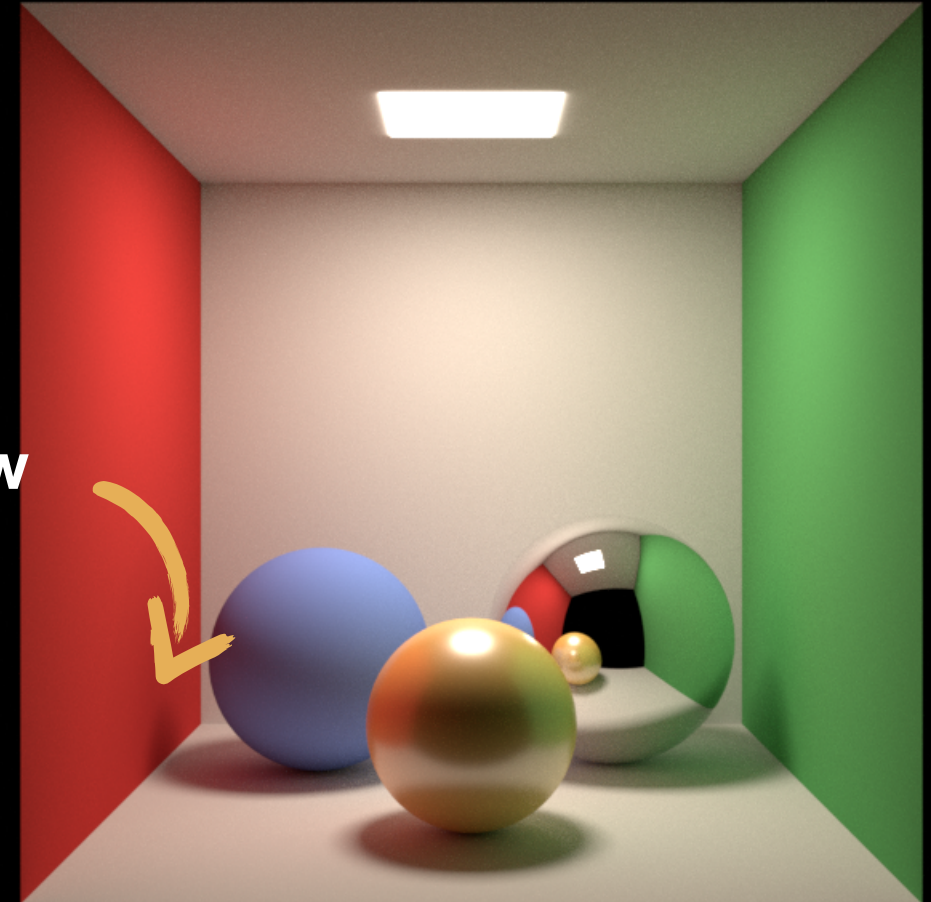
**This lecture: Physically based**

# What is missing?

---



Soft shadow



**Last lecture: Non-physically based**

**This lecture: Physically based**

# Today

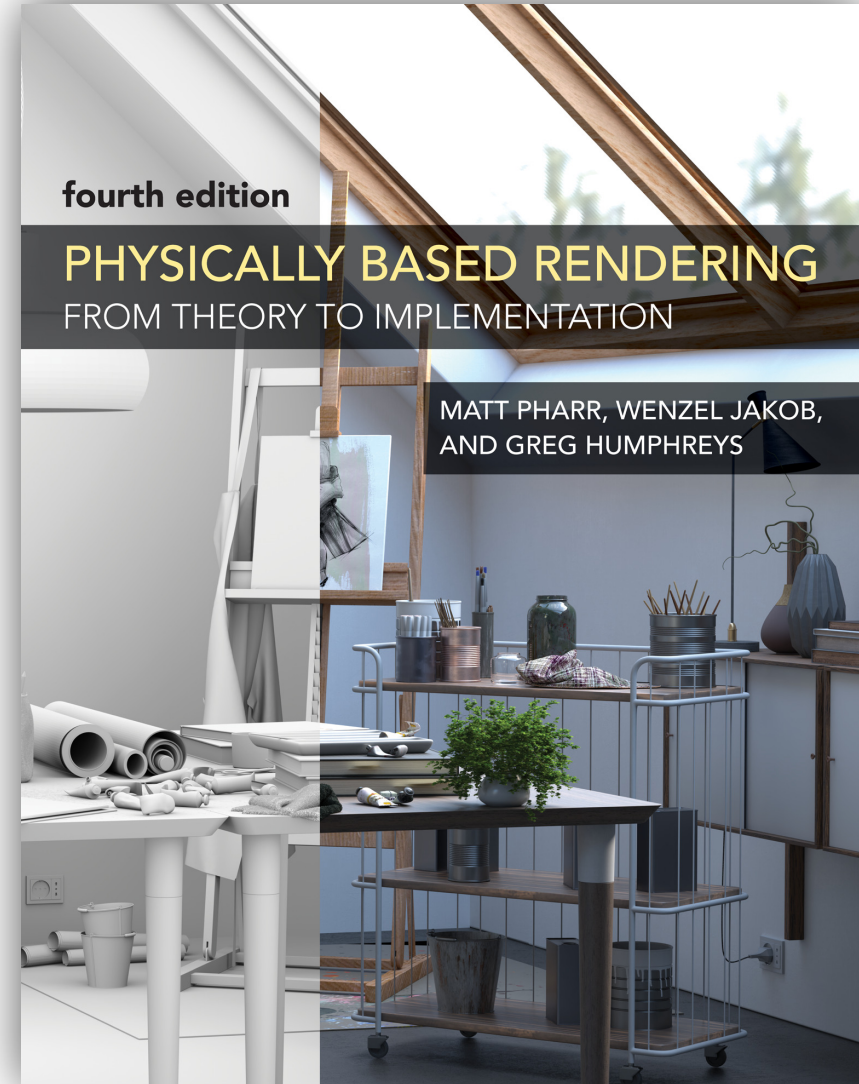
---

- Goal: Understand how a path tracer works
- Disclaimer: I'll skip a lot of physics terms and derivations
- **Not related to assignments or the final**
- The surface scattering equation: what we should compute
- Monte Carlo: how to compute an integral with random samples
- Path tracing: putting 1 and 2 together

# Today

---

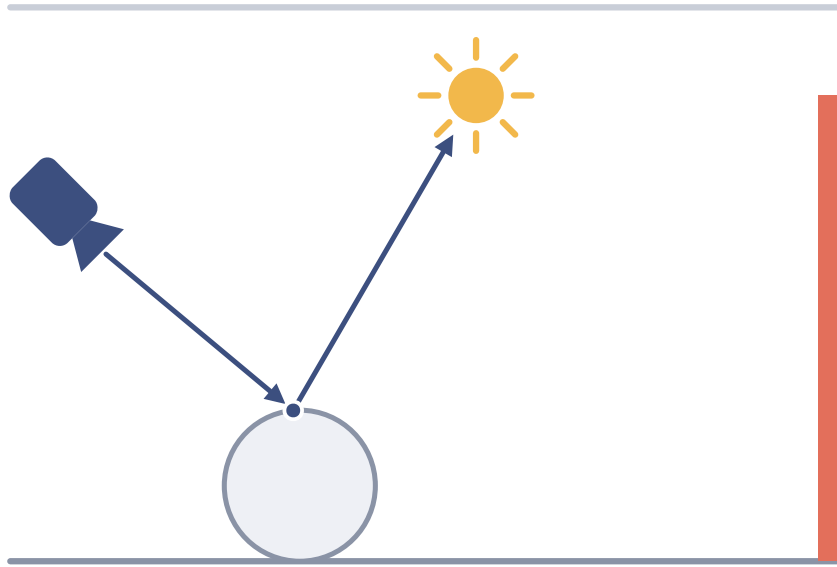
- Reference
  - <https://www.pbr-book.org/>  
Fully free online with code



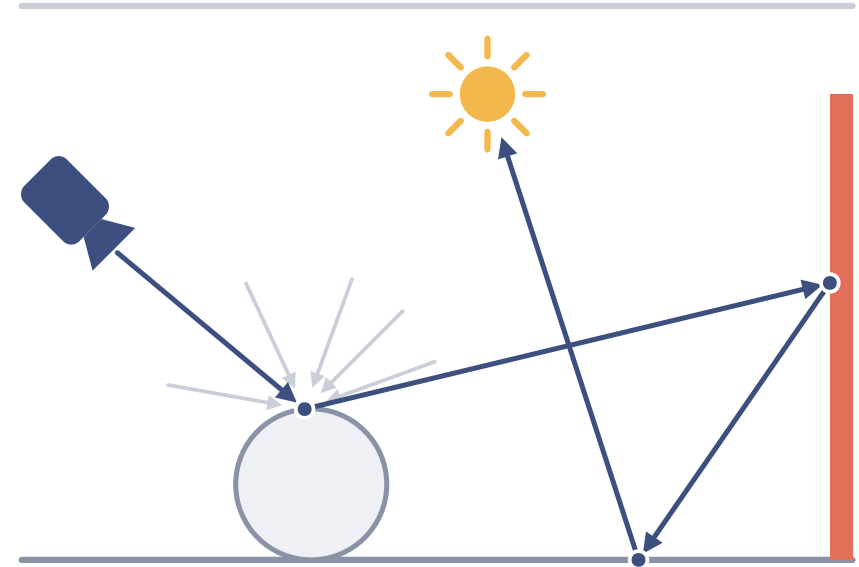
# Surface scattering equation

# What is missing

---



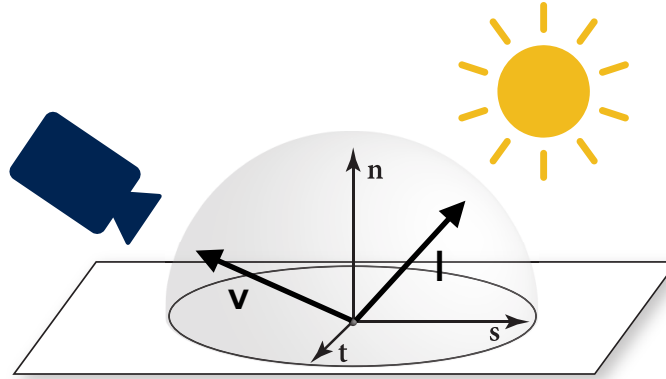
“One bounce”



“Many bounces”

- We need to consider more lights from more directions

# Last lecture: “One bounce”



Pharr, Matt, et al. Physically based rendering. Morgan Kaufmann Publisher, 2016.

diffuse coefficient

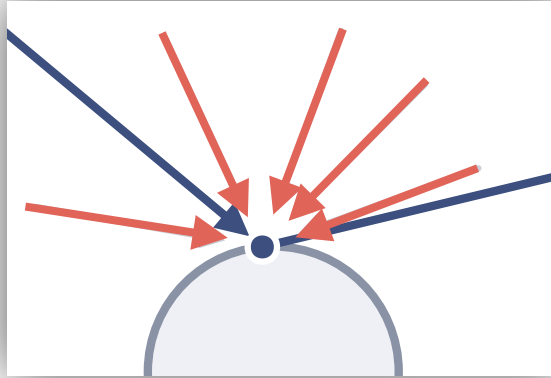
$\cos \theta$  if facing the light

$$L = k_d I \max(0, \mathbf{n} \cdot \mathbf{l})$$

- One light, one direction:  $\mathbf{l}$
- $\cos \theta = \mathbf{n} \cdot \mathbf{l}$ : tilted surfaces receive less light

# Physically based: “Many bounces”

---



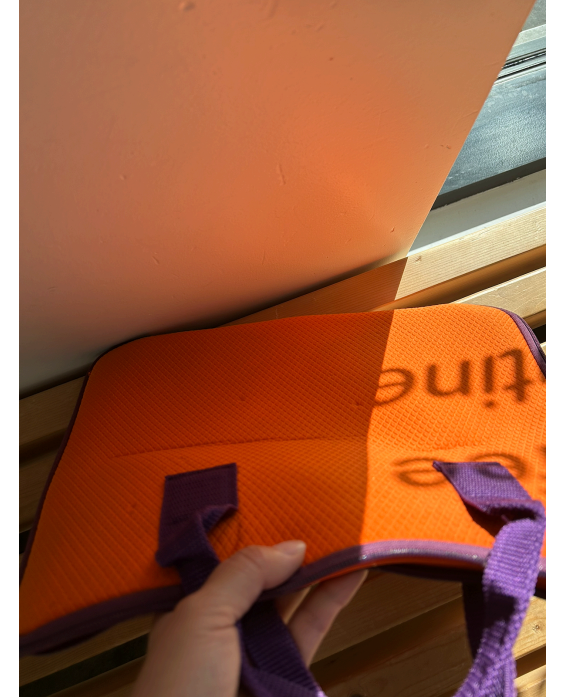
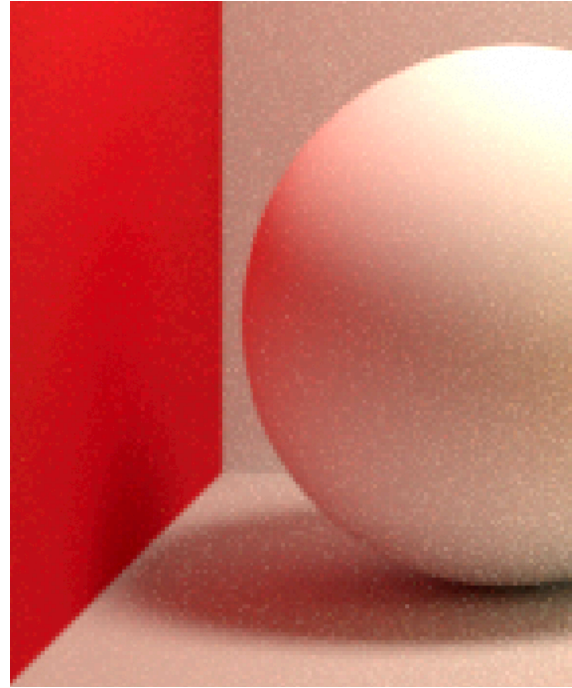
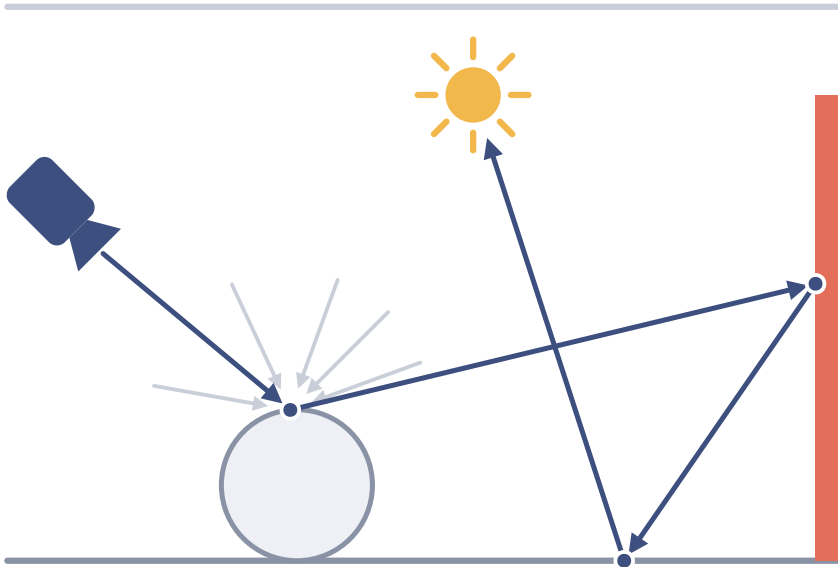
- If you have multiple lights, simply sum them up all together (all the “facing” lights)

$$L = \sum_j k_d I_j \cos \theta_j$$

- Each light: how bright it is ( $I_j$ ), times how much it counts ( $k_d \cos \theta_j$ )

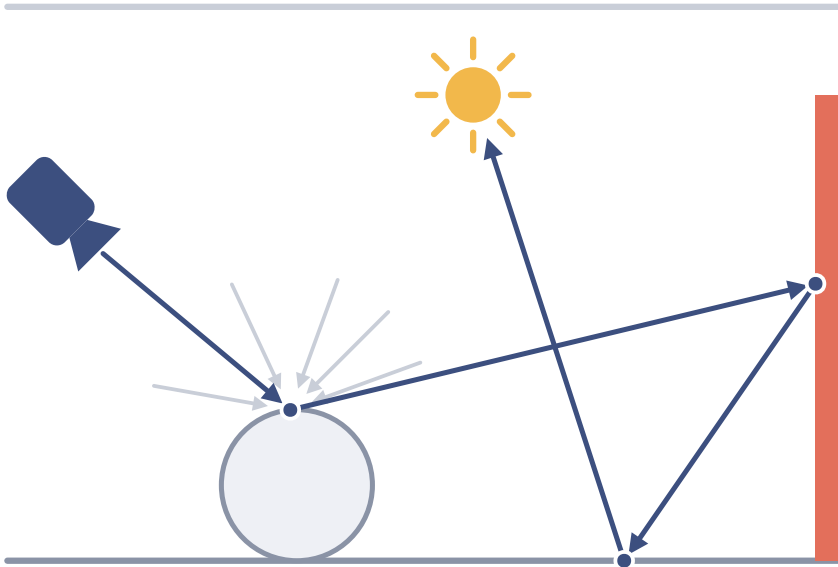
# Every surface is a light

---



- Light arrives from every direction, not only from the lights

# Every surface is a light

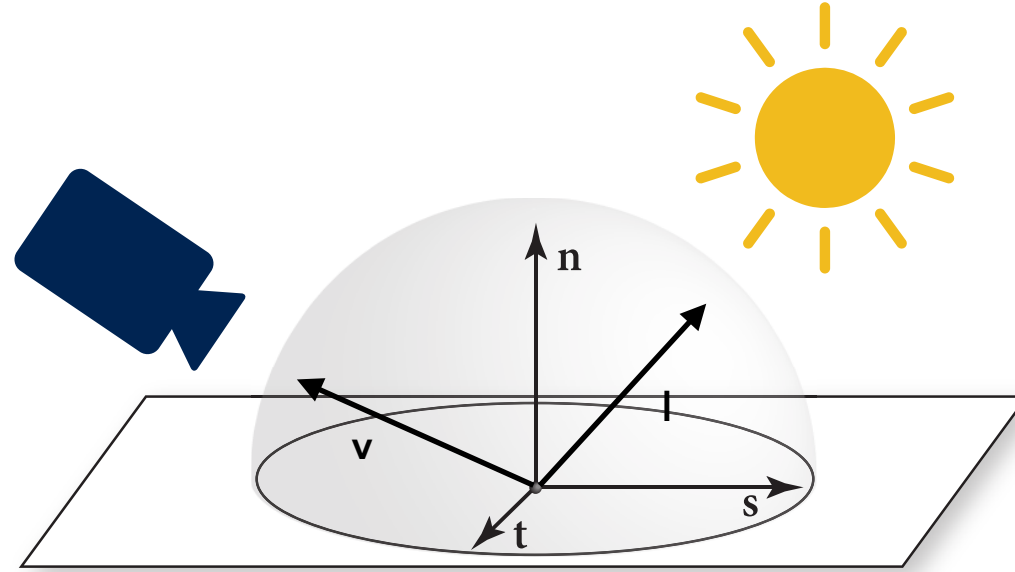


$$L = \sum_j k_d I_j \cos \theta_j$$

$$L = \sum_j k_d L_j(\omega) \cos \theta_j$$

- $I_j \rightarrow L_j(\omega)$ : the brightness of light  $j$  becomes the light arriving from a direction  $\omega$
- $\cos \theta \rightarrow \cos \theta_j$ : the angle between  $\mathbf{n}$  and  $\omega$
- $L_i(\omega) = I_j$  : if points to a light

# From a sum to an integral



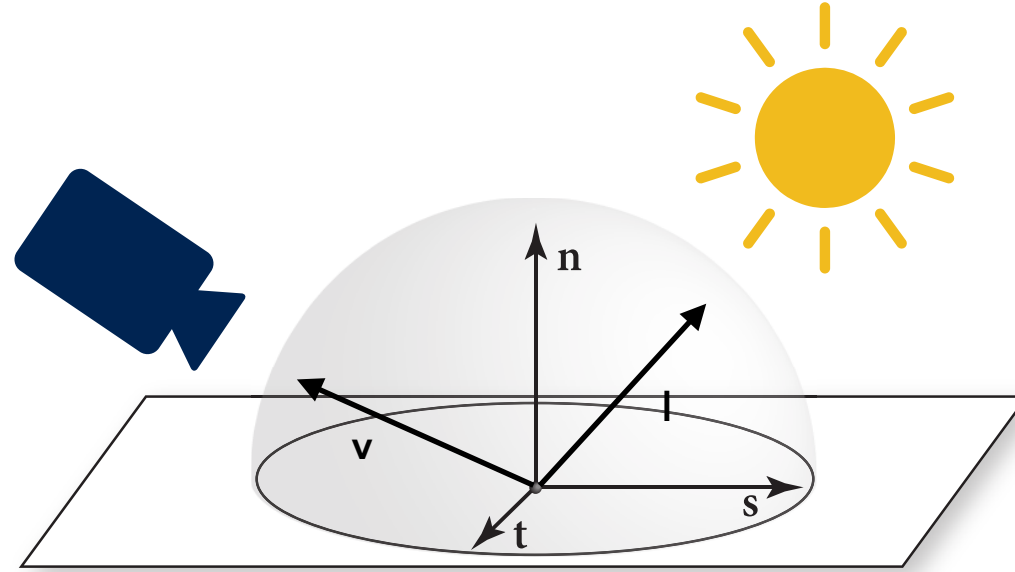
Pharr, Matt, et al. Physically based rendering. Morgan Kaufmann Publisher, 2016.

- Every direction around  $\mathbf{n}$  is a point on the hemisphere
- Sum over all of them:

$$\sum_j \rightarrow \int_{H^2} \dots, d\omega$$

A small bundle of directions

# From a sum to an integral



Pharr, Matt, et al. Physically based rendering. Morgan Kaufmann Publisher, 2016.

$$L = \sum_j k_d L_j(\omega) \cos \theta_j \longrightarrow L = \int_{H^2} k_d L_i \cos \theta d\omega$$

# Surface scattering equation

---

$$L_o = L_e + \int_{H^2} f L_i \cos \theta d\omega$$

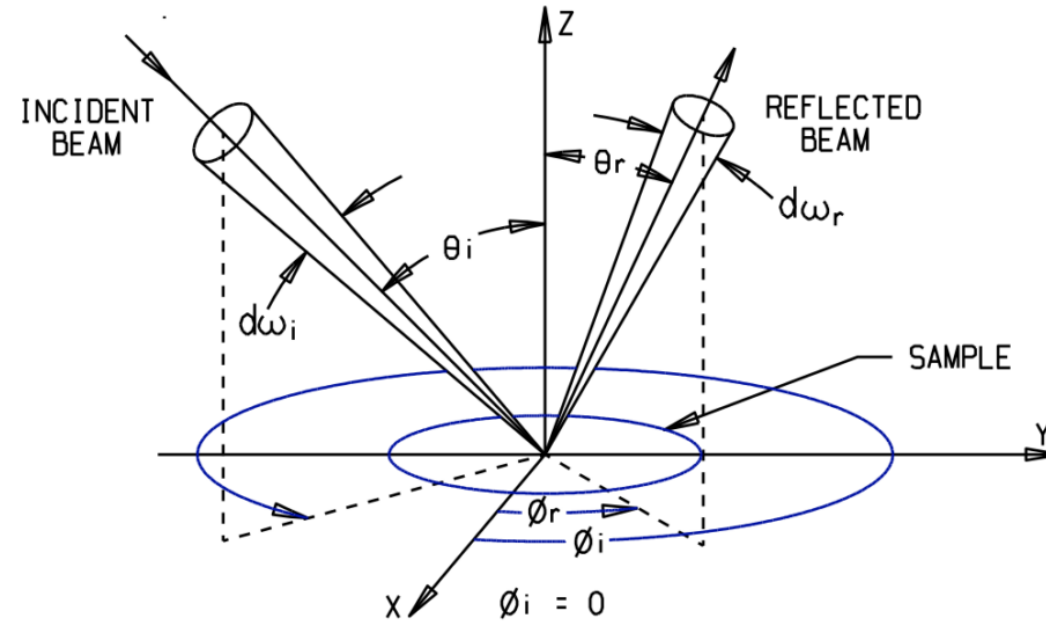
Light the surface emitted  
(e.g., itself is a light)

Replace diffuse coefficient  
(constant) with a function

- Light leaving a point = light it emits + light it reflects
- Gather (  $L_i$  ), weight (  $f \cos \theta$  ), add up (  $\int d\omega$  )

# f: BRDF (Bidirectional Reflectance Distribution Function)

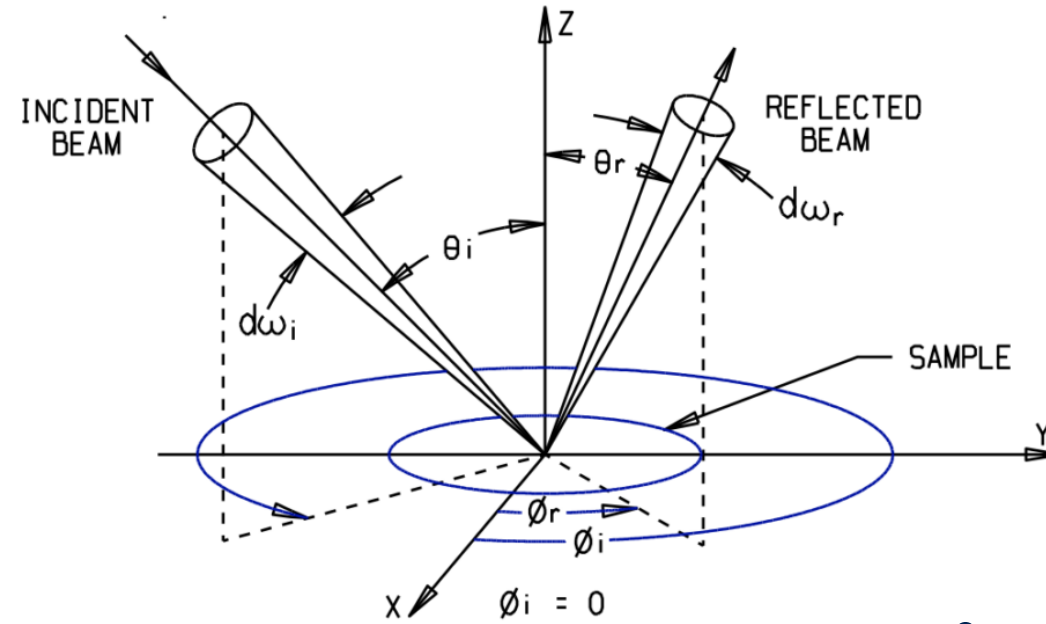
---



<https://surfaceoptics.com/bsdf-brdf-and-btdf-a-review-of-measurement-approaches/>

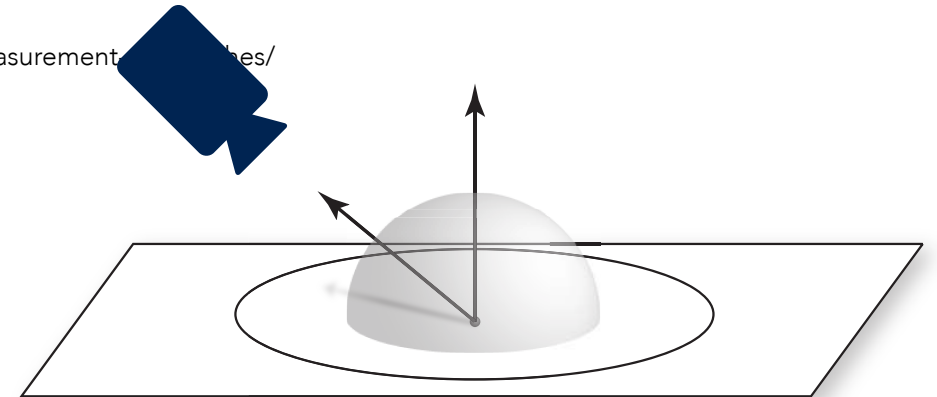
- How much light arriving from one direction leaves toward another

# f: BRDF (Bidirectional Reflectance Distribution Function)

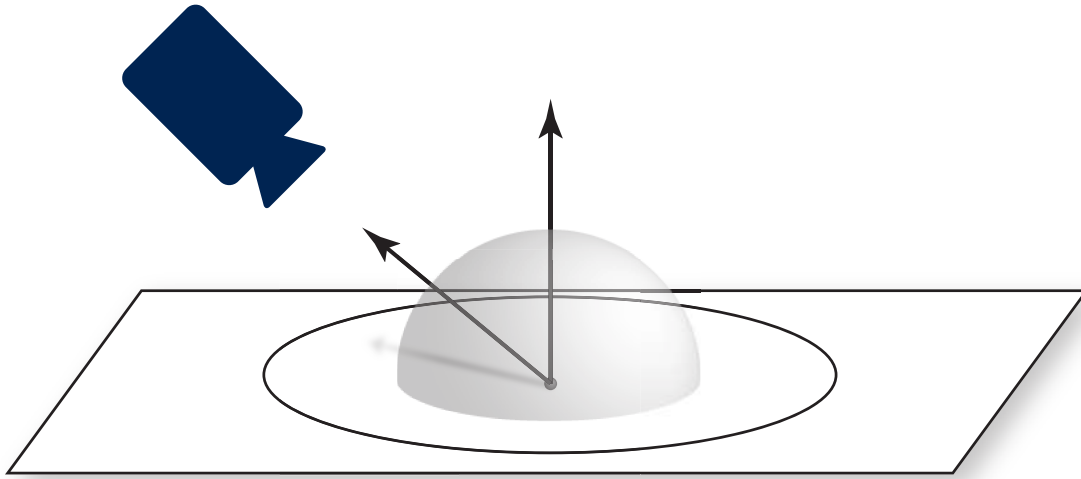


<https://surfaceoptics.com/bsdf-brdf-and-btdf-a-review-of-measurement-techniques/>

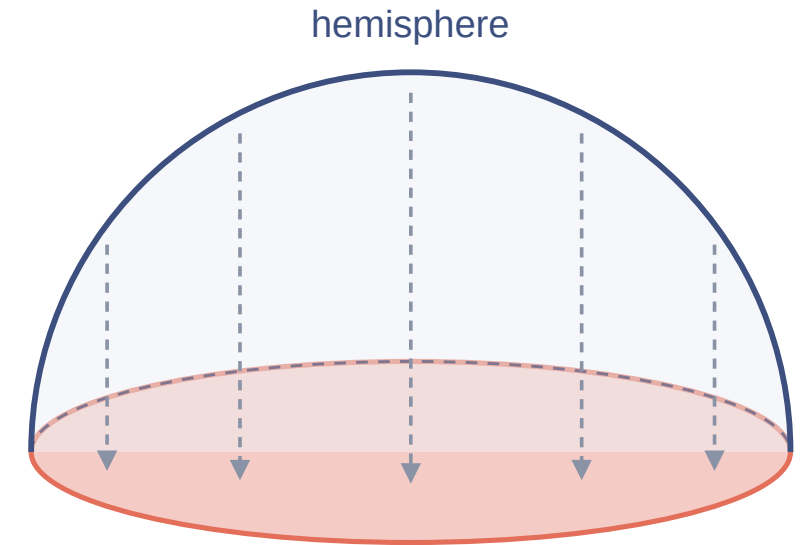
- Diffuse: the same for all directions



# f: BRDF (Bidirectional Reflectance Distribution Function)

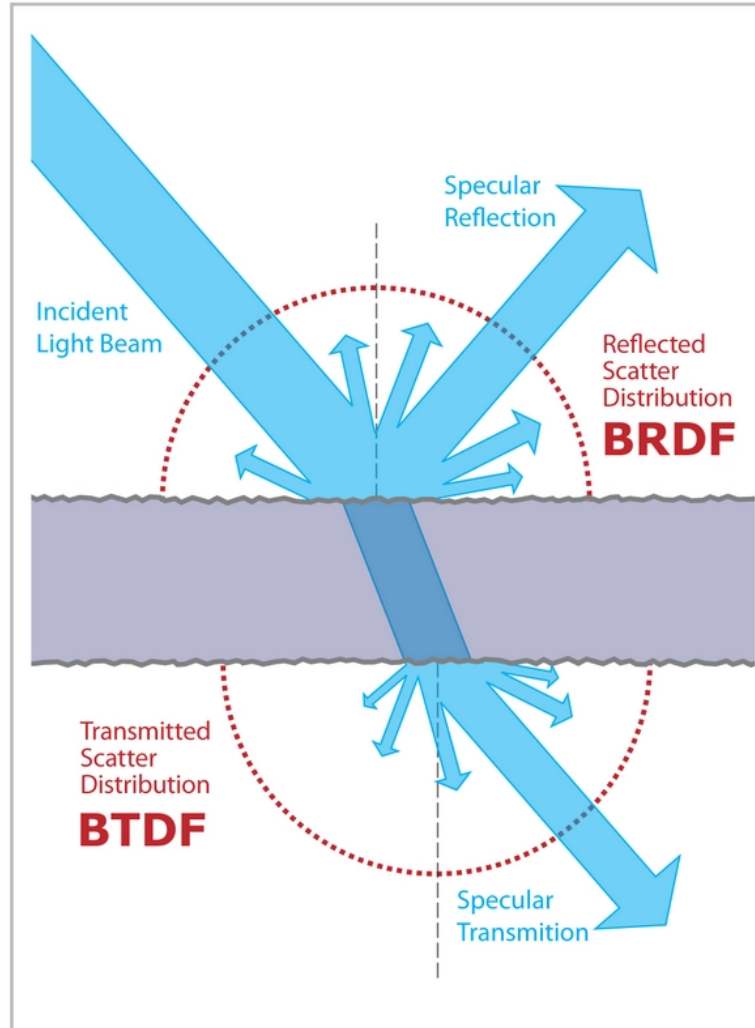


## Why pi?



- Diffuse:  $f = R/\pi$  , albedo:  $R \in [0, 1]$
- R: the fraction of light reflected, per RGB channel
- E.g., the orange bag:  $R = (1, 0.65, 0)$

# BSDF = BRDF + BTDF



[https://en.wikipedia.org/wiki/Bidirectional\\_scattering\\_distribution\\_function#/media/File:BSDF05\\_800.png](https://en.wikipedia.org/wiki/Bidirectional_scattering_distribution_function#/media/File:BSDF05_800.png)

## Blinn-Phong reflection model (ambient + diffuse + specular): a special case

---

$$L = k_a I_a + k_d I \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s I \max(0, \mathbf{n} \cdot \mathbf{h})^p$$

$$L_o = L_e + \int_{H^2} f L_i \cos \theta d\omega$$

- Diffuse coefficient: secretly  $R/\pi$
- Ambient: a constant guess for the rest of the integral
- Shadow ray: tests whether  $L_i$  from the light is blocked
- Mirror:  $f$  is a spike  $\rightarrow$  the integral collapses

# How to compute?

---

$$L_o = L_e + \int_{H^2} f L_i \cos \theta d\omega$$

- Problem 1: The integral has no closed form in general
- Problem 2:  $L_i$  is unknown: it is the light leaving some other surface

# Monte Carlo

# Problem 1: The integral has no closed form in general

---

$$\int_D f(x) dx$$

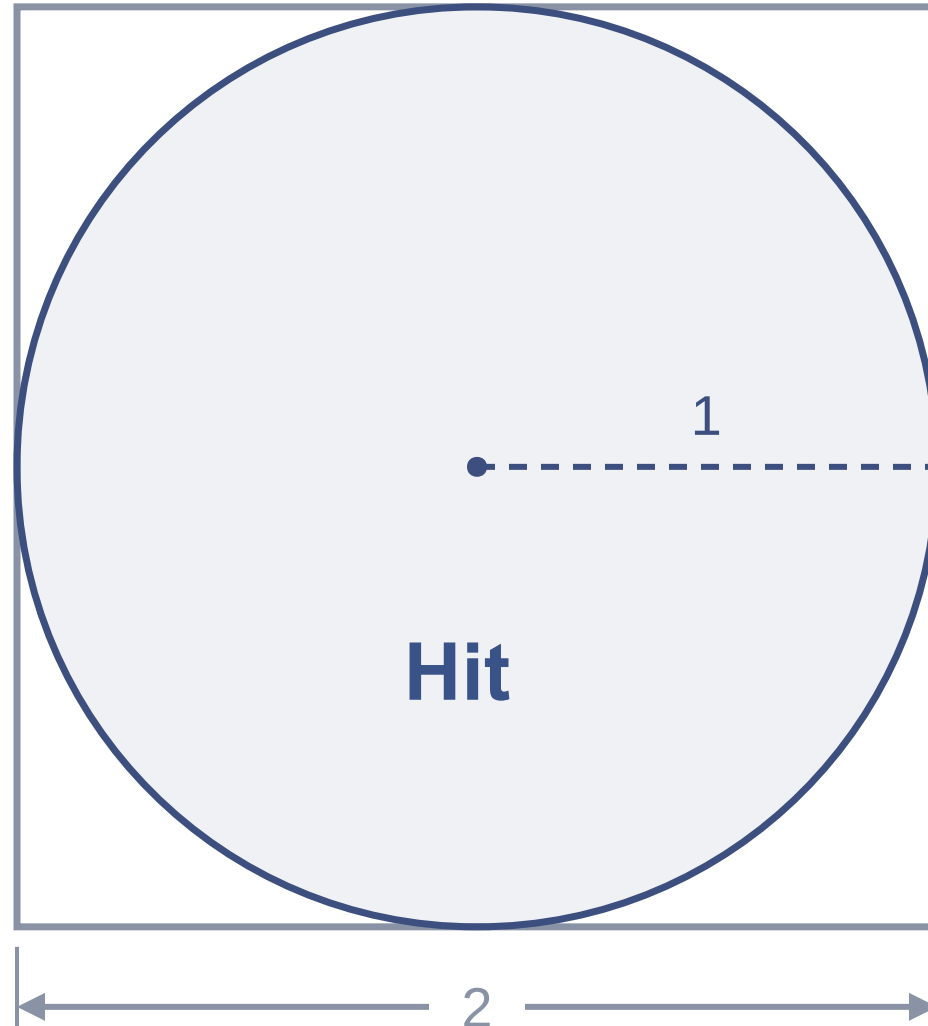
- We can evaluate  $f$  at any point but have no formula for the answer
- One way to integrate: Evaluate  $f$  at many random positions and average

# Estimating pi with darts

---

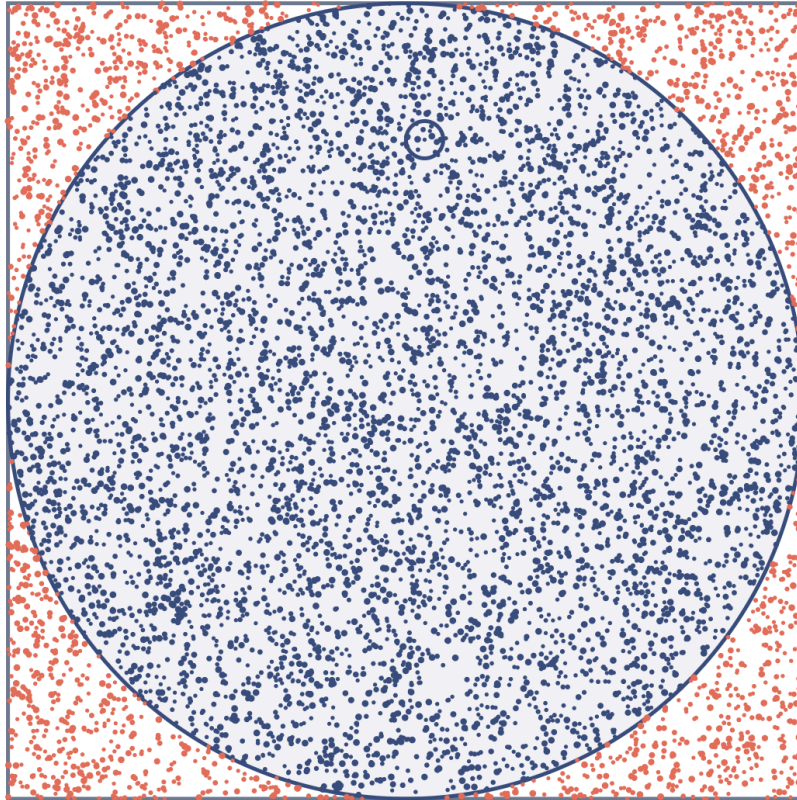


Landing in square  
**uniformly**



$$\pi \approx 4 \cdot \text{hits} / N$$

# Estimating pi with darts



$$\pi \approx 4 \cdot 6,387 / 8,120$$

**3.1463**

DARTS N  
8,120

HITS  
6,387

ERROR  
0.0047

Run

+1

+10

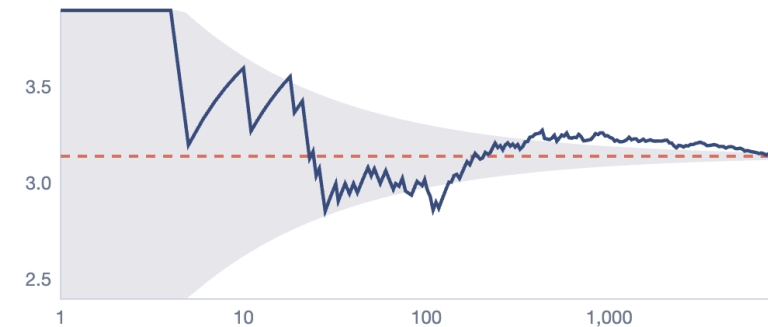
+100

+1,000

Reset

Speed  50 darts/s

ESTIMATE AS N GROWS



— estimate —  $\pi = 3.14159\dots$  typical error  $\approx 1.64 / \sqrt{N}$

Space run / pause · R reset

# Monte Carlo Estimator: Darts

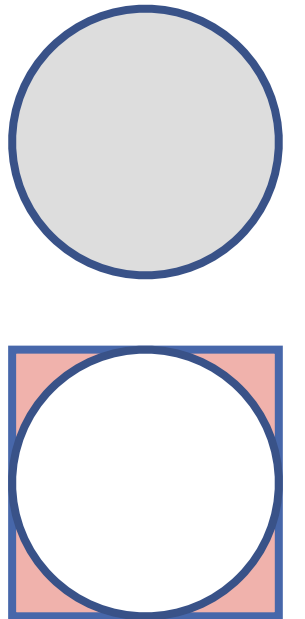
---

$$\pi \approx 4 \cdot \text{hits}/N$$

$$\int f \approx (\text{size of domain}) \times (\text{average of } f \text{ at random points})$$

Square area: 4

$$f(x, y) = \begin{cases} 1 & x^2 + y^2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$



# Monte Carlo Estimator: General form

---

$$\int f \approx \frac{1}{N} \sum_i \frac{f(x_i)}{p(x_i)}$$

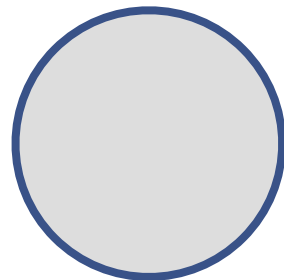
- Sampling probability density function (PDF): Uniform sampling  $p = 1/\text{size}$

# Estimating pi with darts

---

$$\pi = \int_{[-1,1]^2} f(x, y) \, dx \, dy, \quad f(x, y) = \begin{cases} 1 & x^2 + y^2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

- Pi is the area of



# Estimating pi with darts

---

$$\pi = \int_{[-1,1]^2} f(x, y) \, dx \, dy, \quad f(x, y) = \begin{cases} 1 & x^2 + y^2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

- Darts **uniformly** land on the square:

$$p(x, y) = \frac{1}{\text{area of square}} = \frac{1}{4}$$

# Estimating pi with darts

---

$$\pi = \int_{[-1,1]^2} f(x, y) dx dy, \quad f(x, y) = \begin{cases} 1 & x^2 + y^2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

- Plug into the general form:  $\int f \approx \frac{1}{N} \sum_i \frac{f(x_i)}{p(x_i)}$

$$\pi \approx \frac{1}{N} \sum_i \frac{f(x_i, y_i)}{1/4} = 4 \cdot \frac{1}{N} \sum_i f(x_i, y_i)$$

$$\pi \approx 4 \cdot \frac{\text{hits}}{N}$$

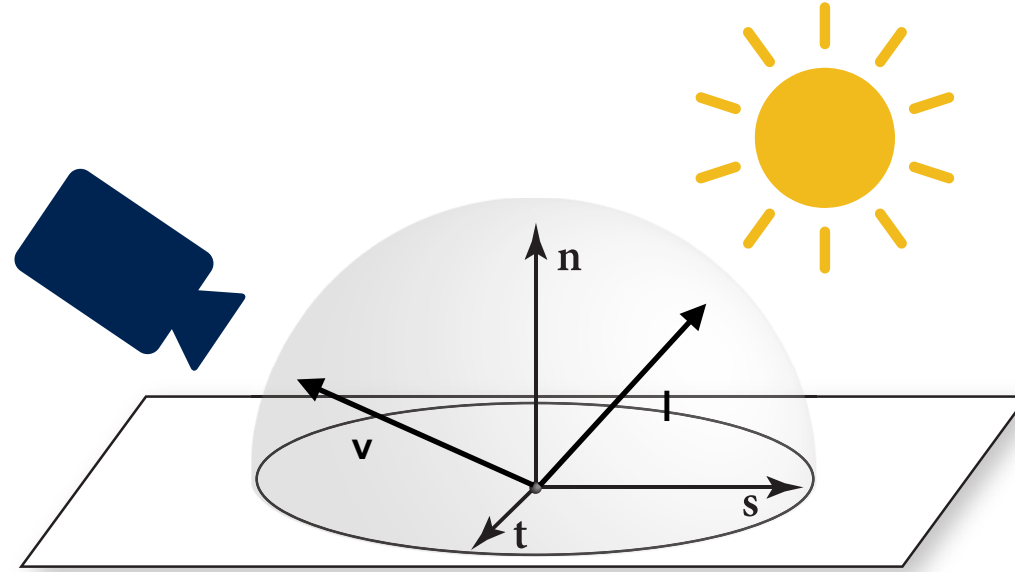
# Proof

---

$$\begin{aligned} E[F_N] &= E \left[ \frac{1}{N} \sum_{i=1}^N \frac{f(X_i)}{p(X_i)} \right] \\ &= \frac{1}{N} \sum_{i=1}^N \int_a^b \frac{f(x)}{p(x)} p(x) dx \\ &= \frac{1}{N} \sum_{i=1}^N \int_a^b f(x) dx \\ &= \int_a^b f(x) dx \end{aligned}$$

- See PBR 4e, Sec. 2.1.3, Eq. 2.7

# Monte Carlo on hemisphere



Pharr, Matt, et al. Physically based rendering. Morgan Kaufmann Publisher, 2016.

- Hemisphere size:  $2\pi$

Over random directions

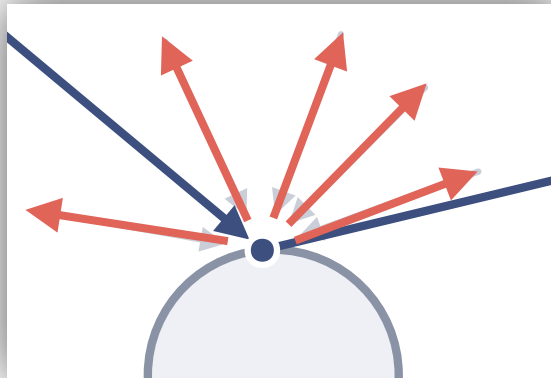
$$L_o \approx L_e + 2\pi \times \text{average of } (f L_i \cos \theta)$$

# Monte Carlo on hemisphere

---

- Problem 2:  $L_i$  is unknown: it is the light leaving some other surface

$$L_o \approx L_e + 2\pi \times \text{average of } (f L_i \cos \theta)$$

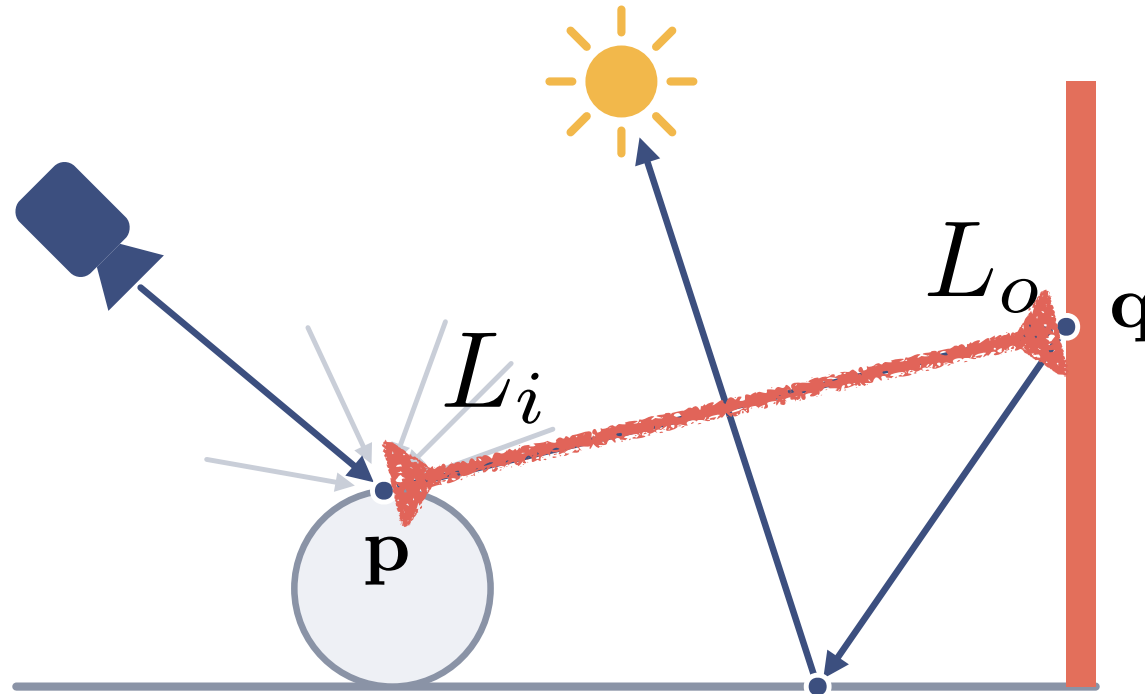


# Path tracing

# $L_i$ is someone else's $L_o$

---

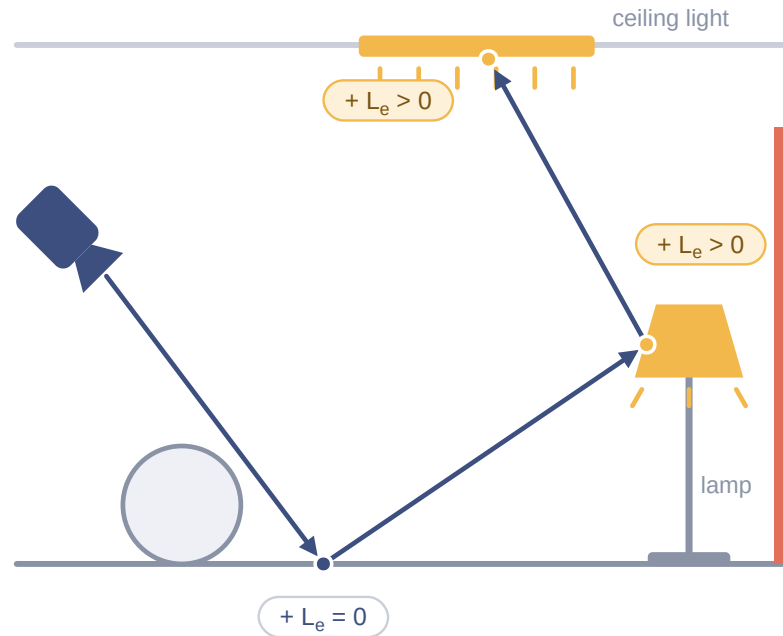
$$L_o \approx L_e + 2\pi \times \text{average of } (f L_i \cos \theta)$$



$$L_i @ p = L_o @ q$$

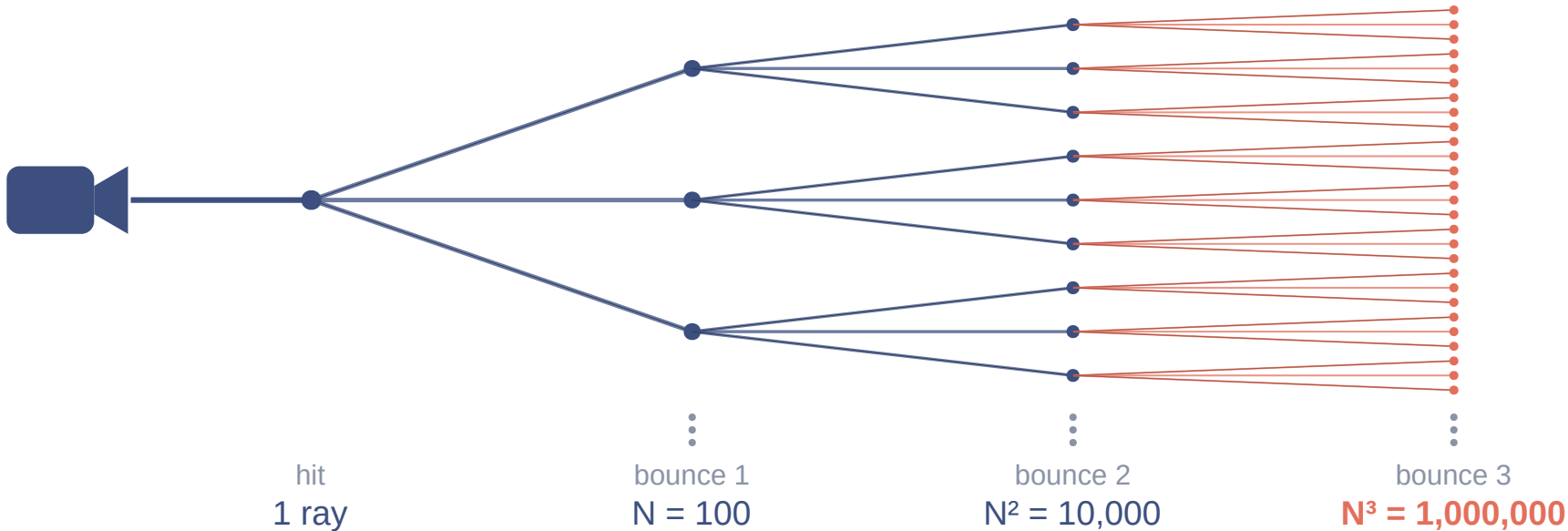
# Lights are surfaces that emit

$$L_o \approx L_e + 2\pi \times \text{average of } (f L_i \cos \theta)$$



- Add  $L_e$  at every hit
- Today: area lights only (emissive surfaces)
- Background color (default black) when not hitting a light

# One sample per bounce



- If we sample  $N$  rays per bounce
- For one pixel, sample  $N=100$  rays per bounce
- At bounce 3, 1,000,000 rays!



# One sample per bounce

---

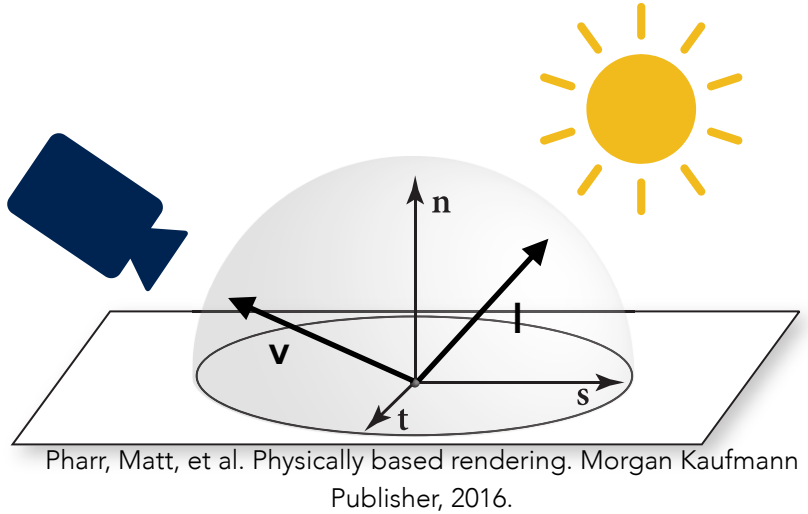
$$\int f \approx \frac{1}{N} \sum_i \frac{f(x_i)}{p(x_i)}$$

- Monte Carlo estimator works even when  $N = 1$

$$\int f \approx \sum_i \frac{f(x_i)}{p(x_i)}$$

# One sample per bounce

---



$$L_o \approx L_e + 2\pi f L_i \cos \theta$$

- $L_i$  is sampled over a uniformly random direction

# Path tracing, recursive version

---

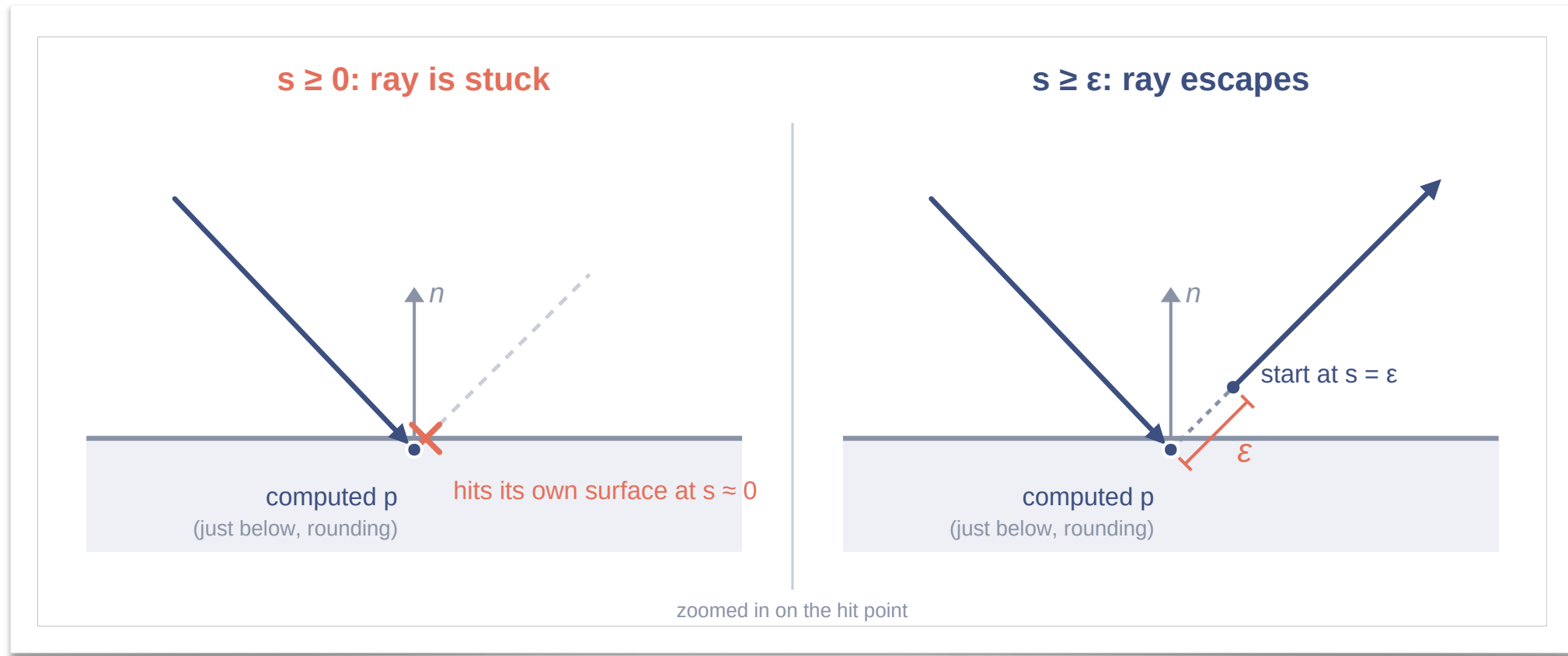
- Same procedure as the mirror recursion

$$L = L_{\text{local}} + k_m \text{raycolor}(\mathbf{p} + s\mathbf{r}), \quad s \in [\epsilon, \infty)$$

Limit the recursion depth!

# Path tracing, recursive version

- Same procedure as the mirror recursion



# Path tracing, recursive version

---

## Mirror reflection

$$L = L_{\text{local}} + k_m \text{raycolor}(\mathbf{p} + s\mathbf{r}), \quad s \in [\epsilon, \infty)$$

Only count  
the local  
emission

Weights based on  
scattering and MC

Random  
direction

$$\text{raycolor}(\text{ray}) = L_e + \frac{f \cos \theta}{p} \text{raycolor}(\text{new ray in direction } \omega)$$

## Path tracing

# Path tracing, recursive version

---

**Plug in PDF and diffuse BRDF**  $p = \frac{1}{2\pi}$   $f = R/\pi$

$$\text{raycolor}(\text{ray}) = L_e + 2R \cos \theta \text{raycolor}(\text{new ray in direction } \omega)$$

# Path tracing, recursive version

---

```
raycolor(ray, depth):  
    if depth > maxDepth: return 0  
    hit = intersect(ray)  
    if no hit: return 0  
     $\omega$  = uniform random direction on the hemisphere around  $n$   
    newRay = (hit.point +  $\epsilon n$ ,  $\omega$ )  
    return  $L_e(\text{hit}) + 2R \cdot \cos\theta \cdot \text{raycolor}(\text{newRay}, \text{depth} + 1)$ 
```

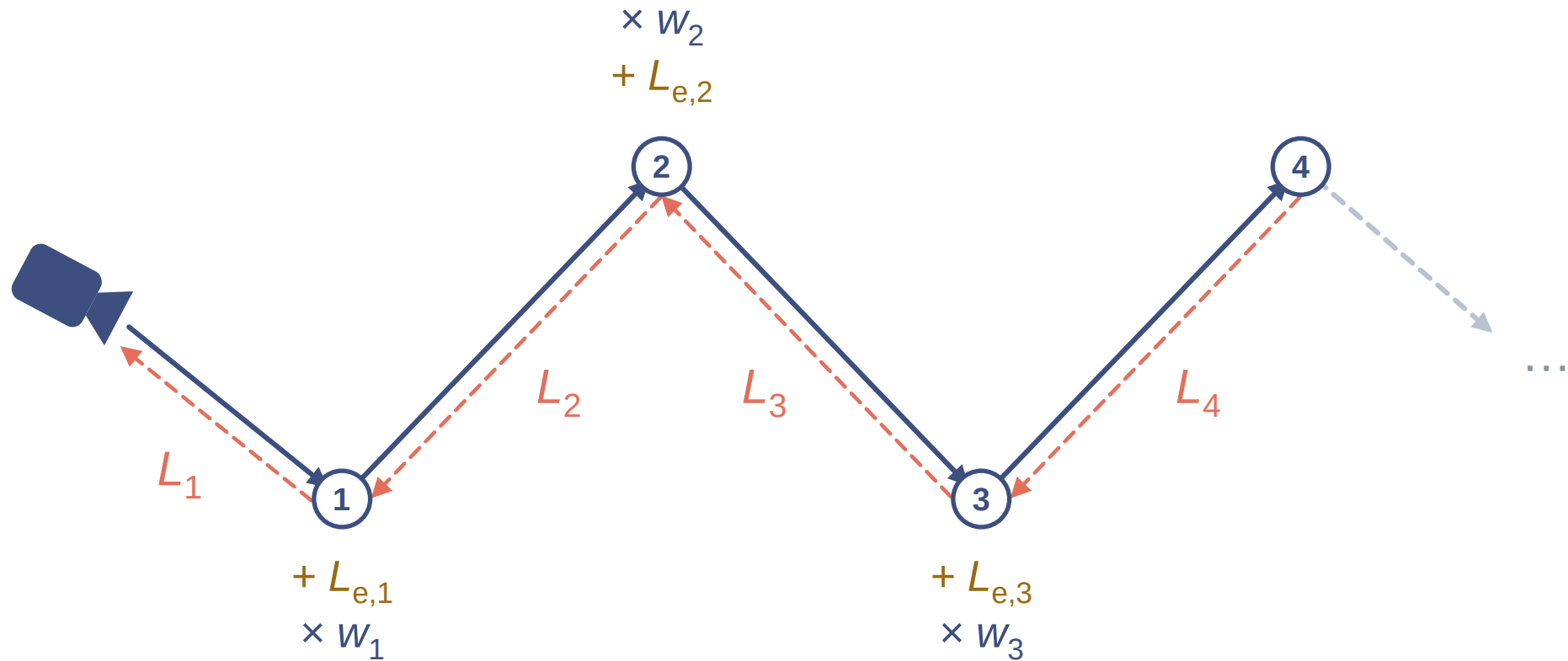
# Path tracing, loop version

---

$$\text{raycolor}(\text{ray}) = L_e + 2R \cos \theta \text{raycolor}(\text{new ray in direction } \omega)$$

$$w = 2R \cos \theta$$

# Path tracing, loop version



$$L_1 = L_{e,1} + w_1 L_2, \quad L_2 = L_{e,2} + w_2 L_3, \quad L_3 = L_{e,3} + w_3 L_4$$

# Path tracing, loop version

---

$$L_1 = L_{e,1} + w_1 L_2, \quad L_2 = L_{e,2} + w_2 L_3, \quad L_3 = L_{e,3} + w_3 L_4$$

- Expand step by step:

$$\begin{aligned} L_1 &= L_{e,1} + w_1 L_2 \\ &= L_{e,1} + w_1 (L_{e,2} + w_2 L_3) = L_{e,1} + w_1 L_{e,2} + w_1 w_2 L_3 \\ &= L_{e,1} + w_1 L_{e,2} + w_1 w_2 (L_{e,3} + w_3 L_4) \\ &= L_{e,1} + w_1 L_{e,2} + w_1 w_2 L_{e,3} + w_1 w_2 w_3 L_4 \\ &= \dots \end{aligned}$$

# Path tracing, loop version

---

- Look at a term in the middle

$$w_1 w_2 \dots w_{i-1} L_{e,i}$$

- Define a new variable:

$$\beta_i = w_1 w_2 \dots w_{i-1}$$

$$\begin{aligned} L_1 &= L_{e,1} + w_1 L_2 \\ &= L_{e,1} + w_1 (L_{e,2} + w_2 L_3) = L_{e,1} + w_1 L_{e,2} + w_1 w_2 L_3 \\ &= L_{e,1} + w_1 L_{e,2} + w_1 w_2 (L_{e,3} + w_3 L_4) \\ &= L_{e,1} + w_1 L_{e,2} + w_1 w_2 L_{e,3} + w_1 w_2 w_3 L_4 \\ &= \dots \end{aligned}$$

# Path tracing, loop version

---

- Look at a term in the middle

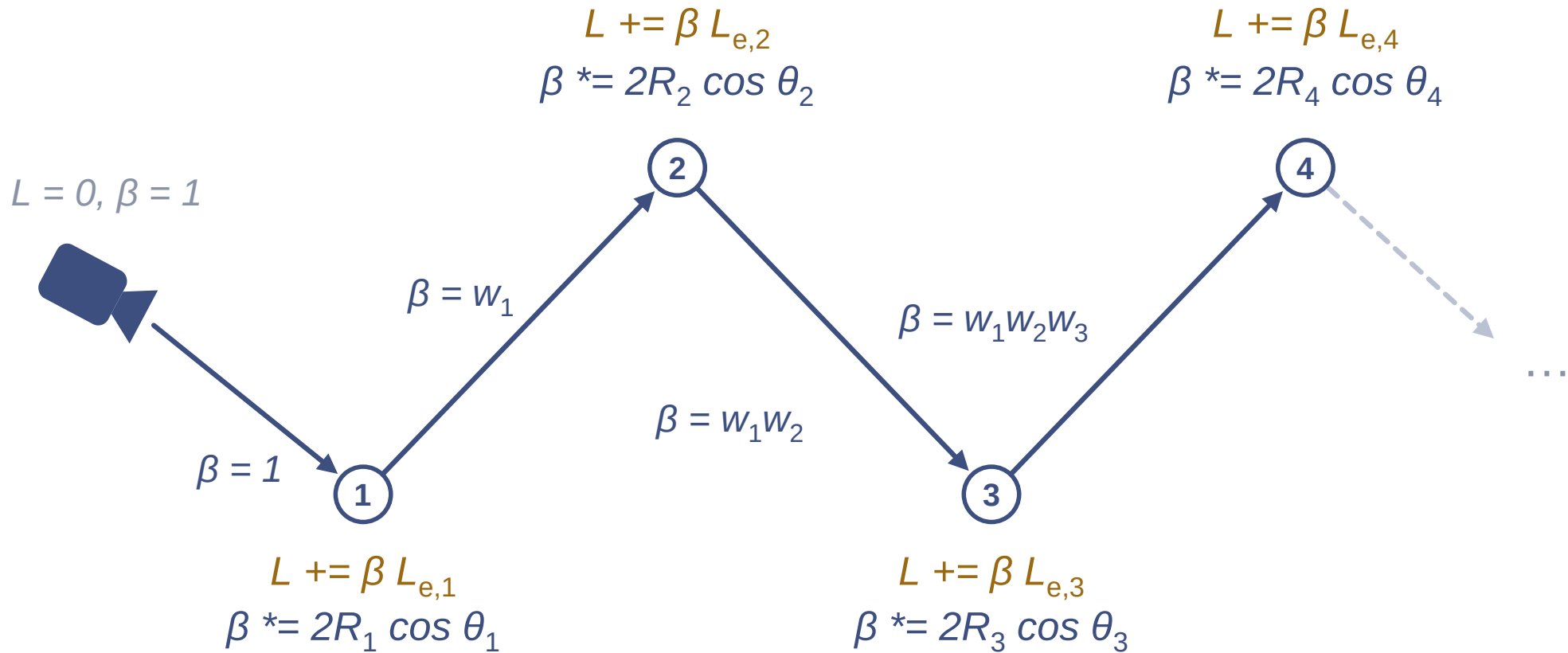
$$w_1 w_2 \dots w_{i-1} L_{e,i}$$

- Define a new variable:

$$\beta_i = w_1 w_2 \dots w_{i-1}$$

$$L_1 = \sum_i \beta_i L_{e,i}, \quad \beta_1 = 1$$

# Path tracing, loop version

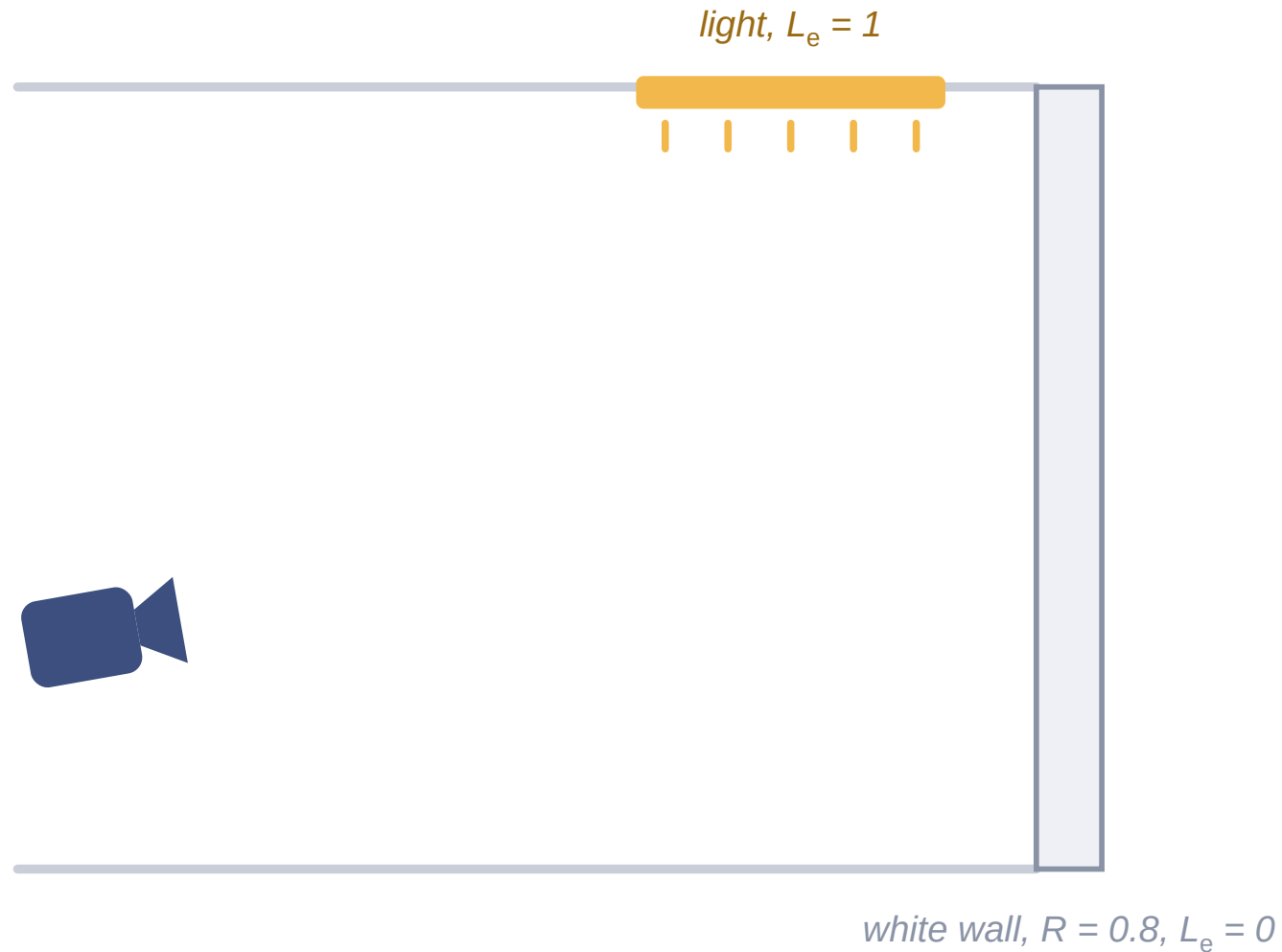


# Path tracing, loop version

```
raycolor(ray):  
     $L = 0$ ;  $\beta = 1$   
    for depth in  $0..maxDepth$ :  
        hit = intersect(ray)  
        if no hit: break  
         $L += \beta \cdot Le(hit)$   
         $\omega = \text{uniform random direction on the hemisphere around } n$   
         $\beta *= 2R \cdot \cos\theta$   
        ray = (hit.point +  $\epsilon n$ ,  $\omega$ )  
    return  $L$ 
```

Max depth: simple, slightly biased

# Toy example

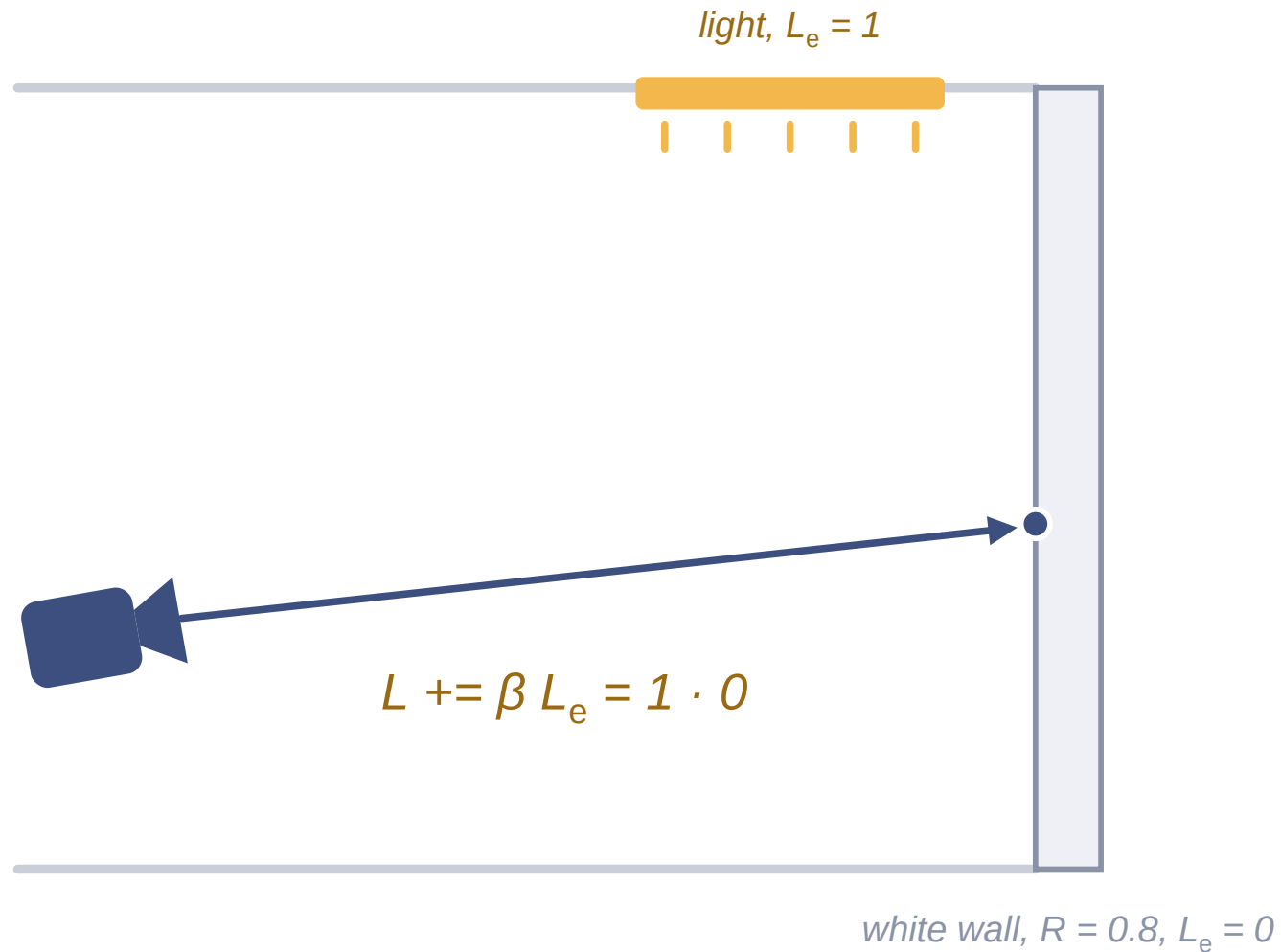


LOOP STATE

$$L = 0$$

$$\beta = 1$$

# Toy example

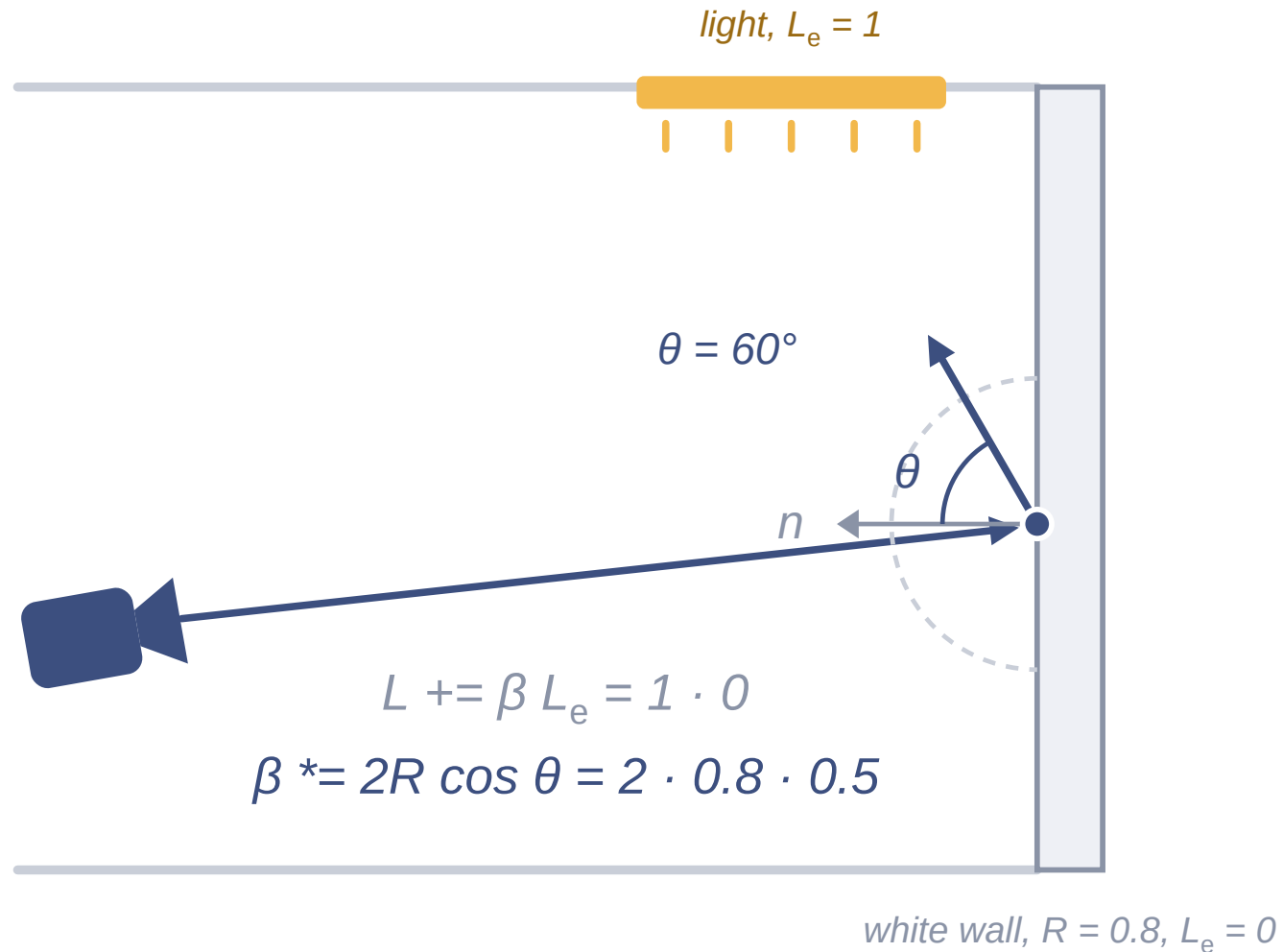


LOOP STATE

$$L = 0$$

$$\beta = 1$$

# Toy example

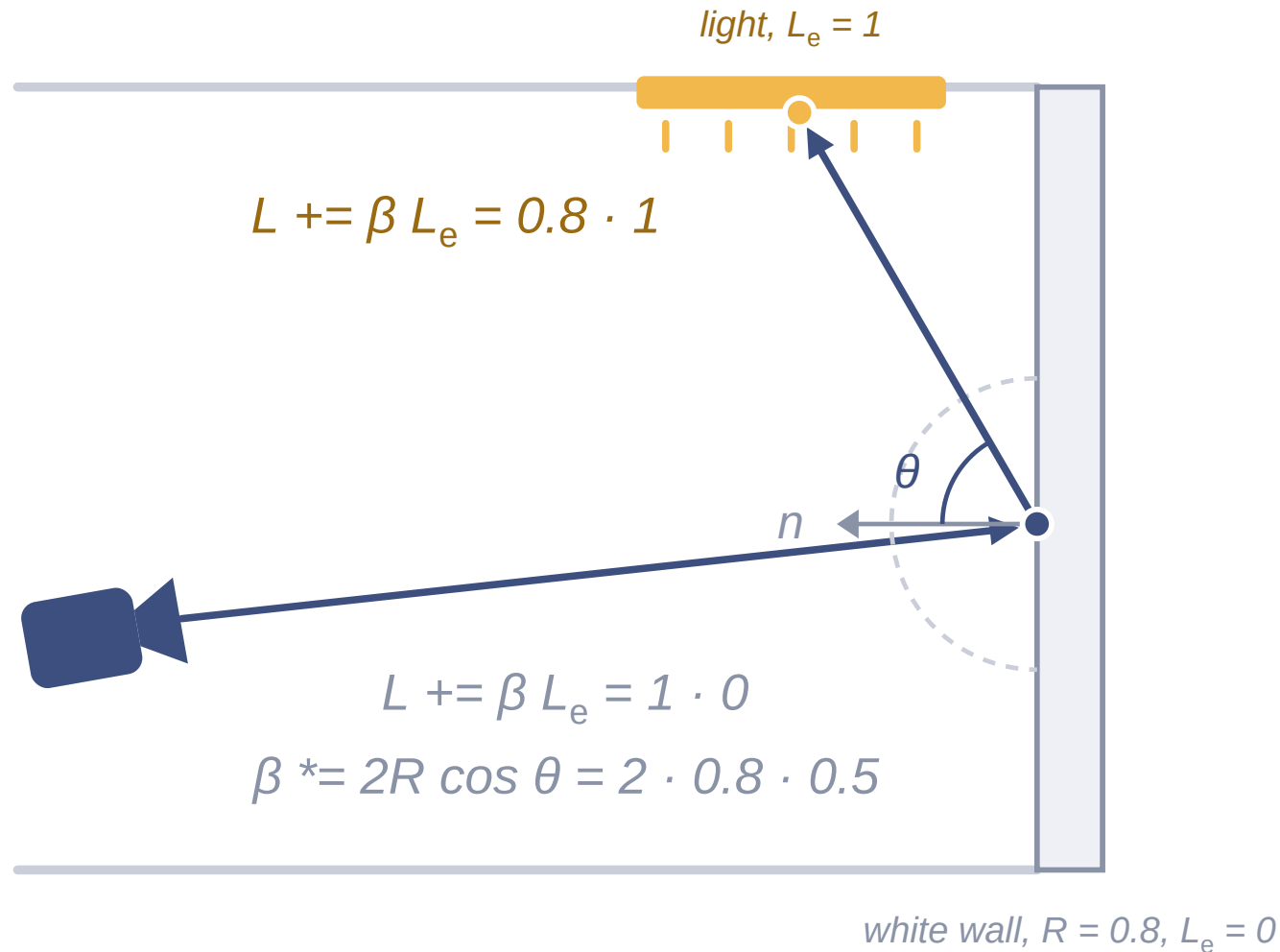


LOOP STATE

$$L = 0$$

$$\beta = 0.8$$

# Toy example

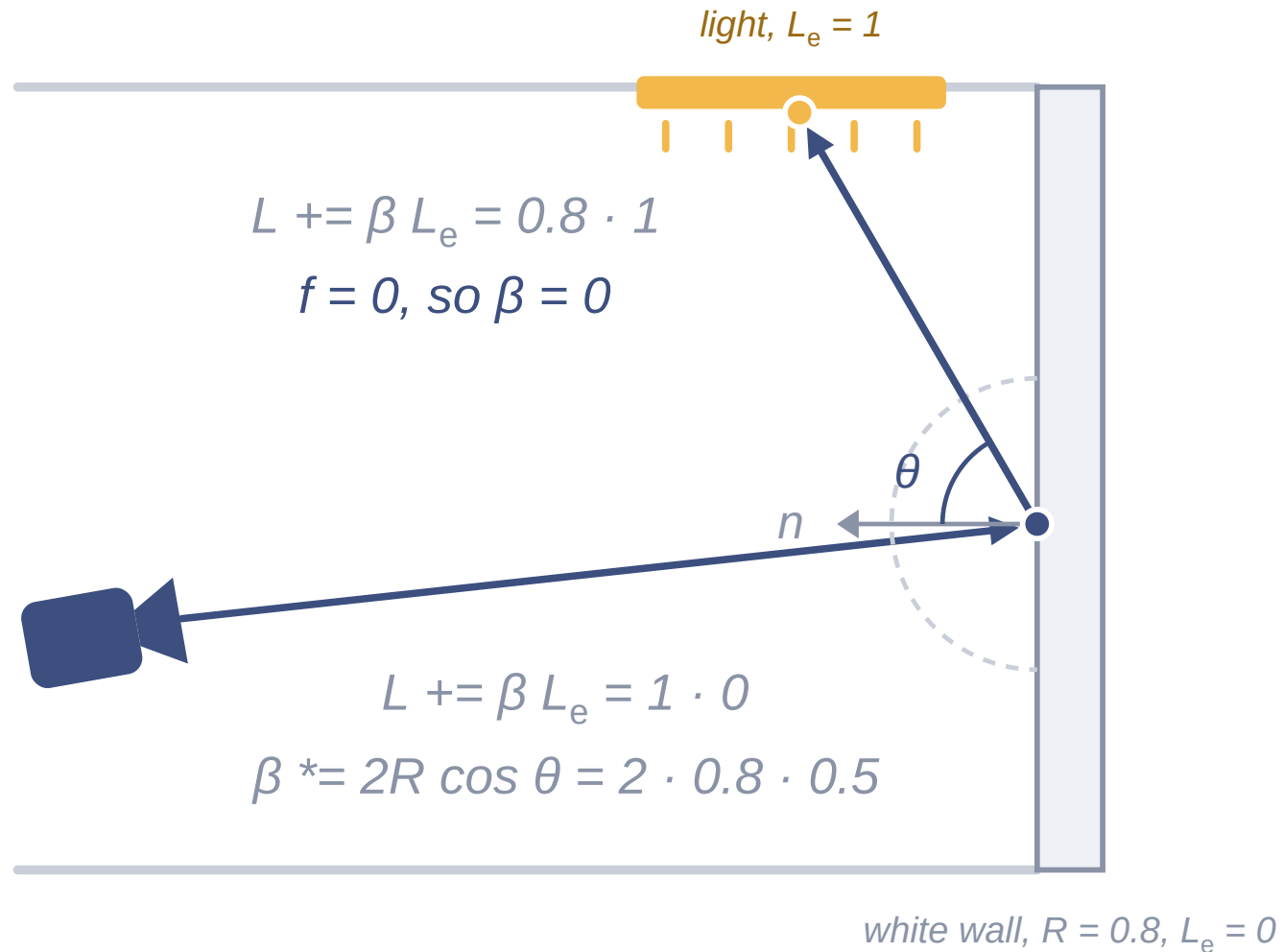


LOOP STATE

$$L = 0.8$$

$$\beta = 0.8$$

# Toy example



LOOP STATE

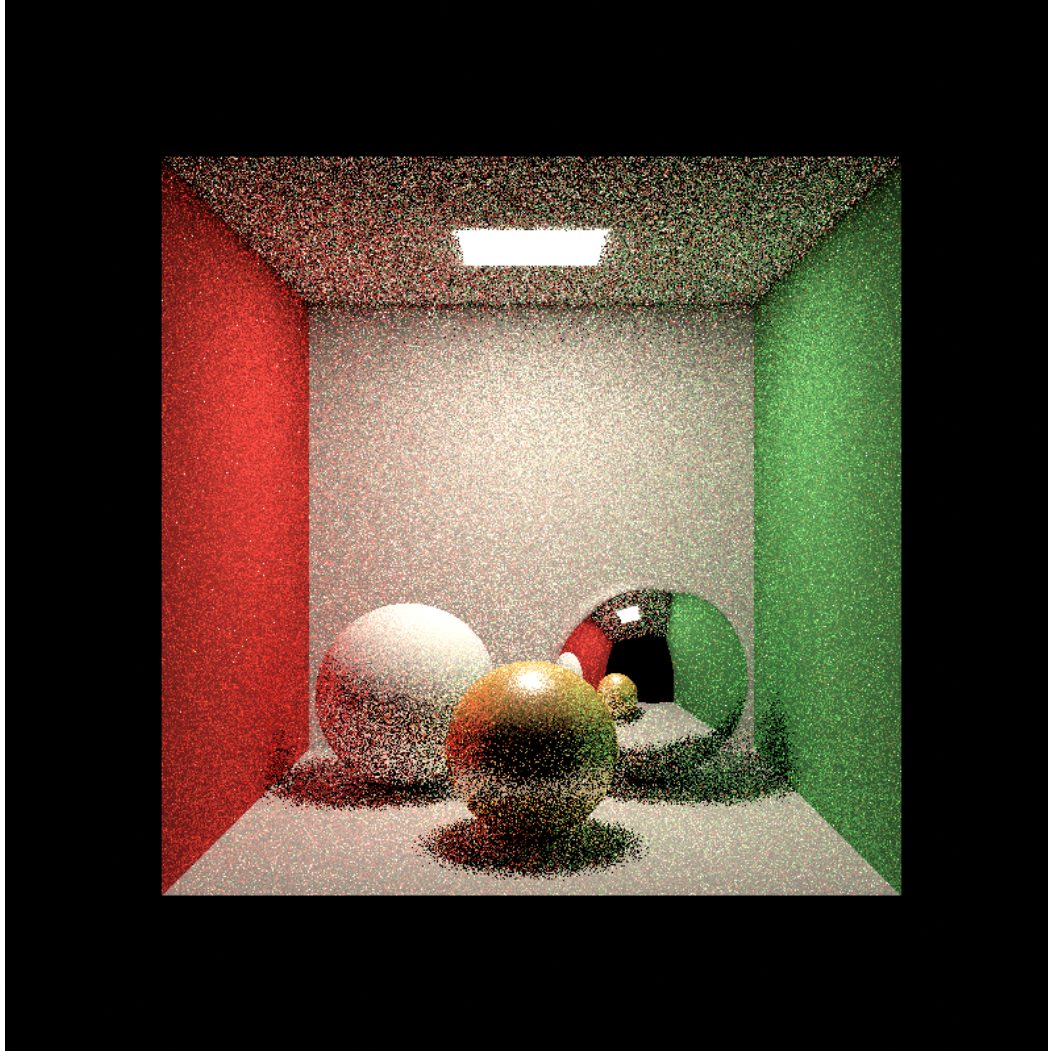
$$L = 0.8$$

$$\beta = 0$$

**Put it together**

# Image is noisy

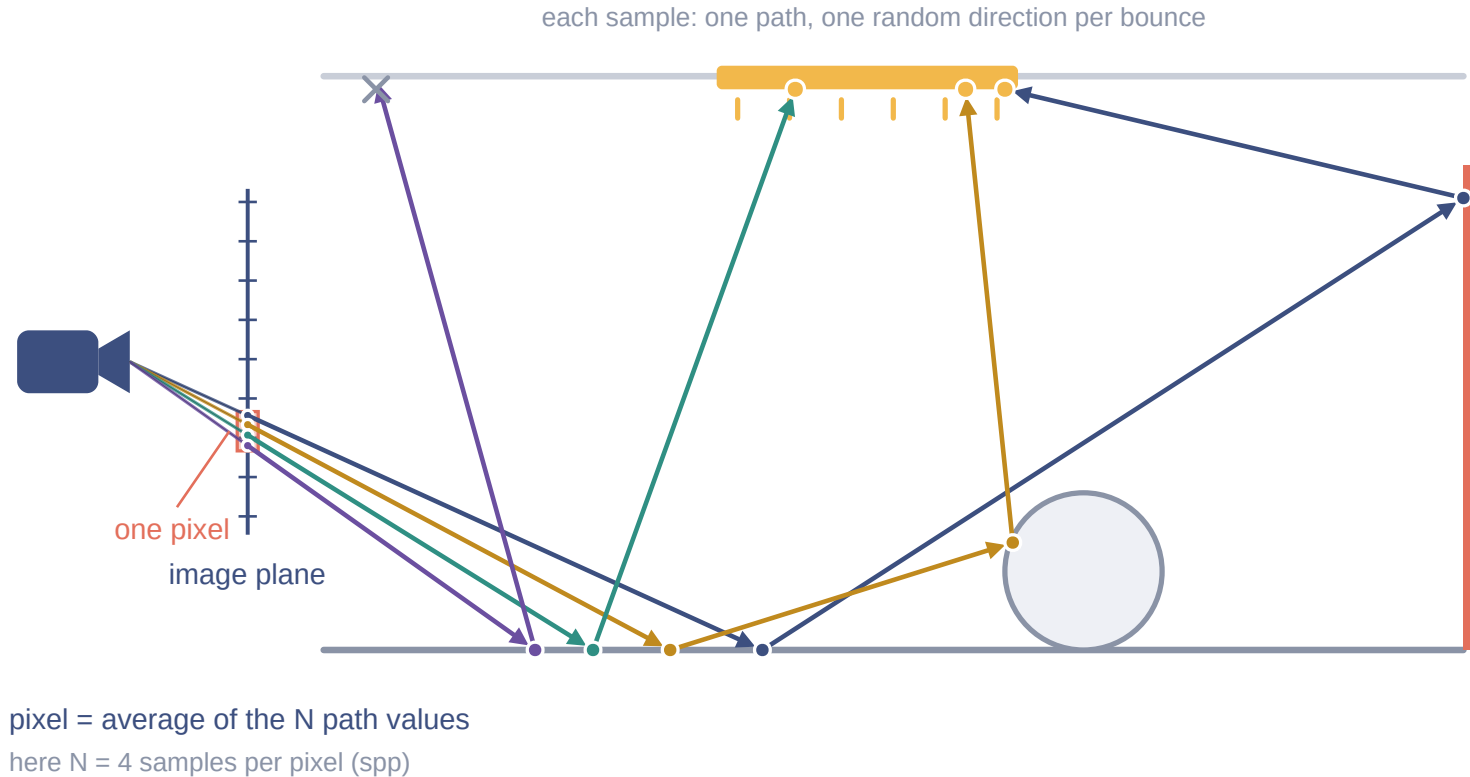
---



$$\int f \approx \sum_i \frac{f(x_i)}{p(x_i)}$$

is noisy!

# Fix: average many paths per pixel



- Spp (samples per pixel): Use a single path per sample but sample N paths per pixel
- This also gives anti-aliasing

# Fix: average many paths per pixel

---

```
for each pixel:  
    color = 0  
    for s in 1..spp:  
        ray = camera ray through a random point in the pixel  
        color += raycolor(ray)  
    pixel = color / spp
```

# What real renderers do next

---

- **Smarter sampling** (less noise for the same spp): aim rays at lights, which also handles point lights; pick directions where  $f \cos$  is large
- **Richer materials**: metals, glossy plastics, glass, skin and marble (light entering below the surface)
- **Volumes**: fog, smoke, clouds, where light scatters inside a medium, not only at surfaces
- **Harder light paths**: caustics (light focused through glass or water) need algorithms beyond plain path tracing
- **Speed**: *acceleration structures*, GPU ray tracing hardware
- **ML denoising**: a neural network turns a noisy low-spp image into a clean one. Blender ships Intel Open Image Denoise and NVIDIA OptiX; games use NVIDIA DLSS Ray Reconstruction
- **Neural and differentiable rendering**: learn a scene from photos (NeRF, 3D Gaussian splatting), or run the renderer backward to recover shape and materials from images (inverse rendering)

# Summary

---

- Surface scattering equation: what to compute
  - Diffuse BRDF:  $f = R/\pi$ ,  $R$  per RGB channel
  - Blinn-Phong: the same idea, summed over the lights only

# Summary

---

- Monte Carlo: how to compute any integral

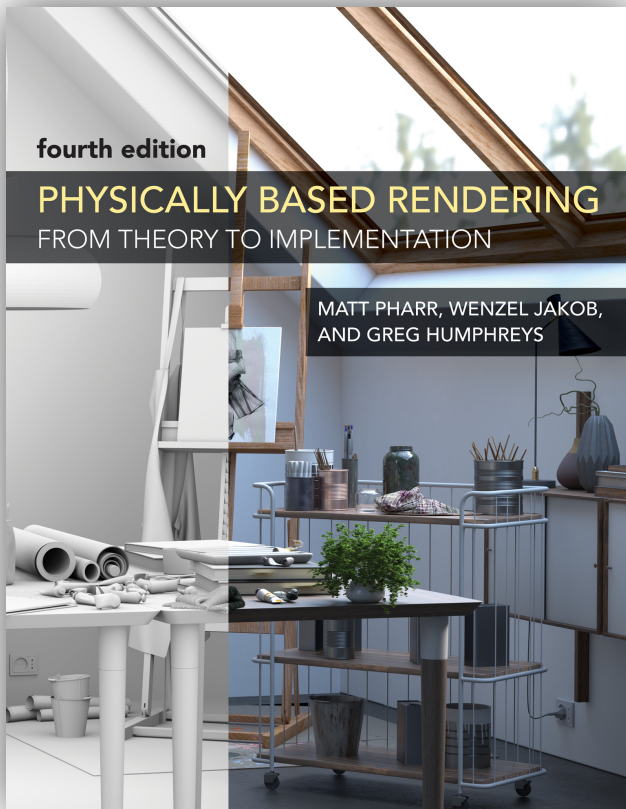
$$\int f(x) dx \approx \frac{1}{N} \sum_{i=1}^N \frac{f(X_i)}{p(X_i)}, \quad \text{noise} \propto \frac{1}{\sqrt{N}}$$

$$p = \frac{1}{2\pi} \text{ over a hemisphere} \quad L_o \approx L_e + 2\pi f L_i \cos \theta$$

(N = 1)

# Summary

- Path tracing



```
raycolor(ray):  
     $L = 0$ ;  $\beta = 1$   
    for depth in 0..maxDepth:  
        hit = intersect(ray)  
        if no hit: break  
         $L += \beta \cdot Le(\text{hit})$   
         $\omega = \text{random direction around } n$   
         $\beta *= f \cdot \cos\theta / p$   
        ray = (hit.point +  $\epsilon n$ ,  $\omega$ )  
    return  $L$   
  
for each pixel:  
    pixel = average of raycolor(  
        camera ray through a random point in the pixel  
    ) over spp paths
```